

Montar case passo a passo

Montar case passo a passo

Date exported: 18/08/2026, 21:17:10



preciso de sua ajuda , preciso que voce me de um passo a passo de como eu posso montar esse case para a vaga que eu estou concorrendo , quero que voce leia todo o case , prepare etapas , escolha a soluç o mais facil .. me oriente em todo processo , cumpra todos os requisitos ... quero que voce seja meu instrutor , me guia desde o zero ate o final , me ajude a criar arquivos ... subir no githubb , gerar codigos para mim , fazer tudo bem certinho , nos m nimos detalhes possiveis , de forma bem facil

Ol , Bernnardo! Tudo bem?

Parab ns! Voc  foi selecionado(a) para a etapa de case referente ao processo seletivo para a vaga de Est gio em Technology.

As instru  es e coordenadas para realizar o case encontram-se no arquivo anexo.

Para a entrega:

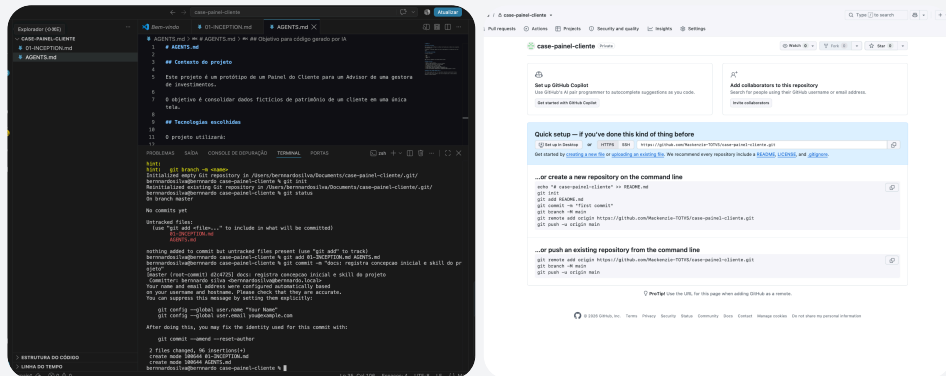
- Deve-se criar reposit rio ****privado**** no GitHub, com acesso de leitura concedido ao usu rio **rodfercar**.
- Preserve o hist rico de commits. N o fa a squash de tudo em um  nico commit nem force-push que apague a evolu  o do trabalho.
- Inclua um [README.md](#) na raiz apontando onde est  cada artefato
- Responda este email com o endere o do GitHub e caso sinta dificuldade de fazer o upload, fique a vontade para compartilhar o anexo conosco atrav s deste email.
- Prazo: ****20 de agosto (quinta-feira)****.

Qualquer d vida, estamos   disposi  o

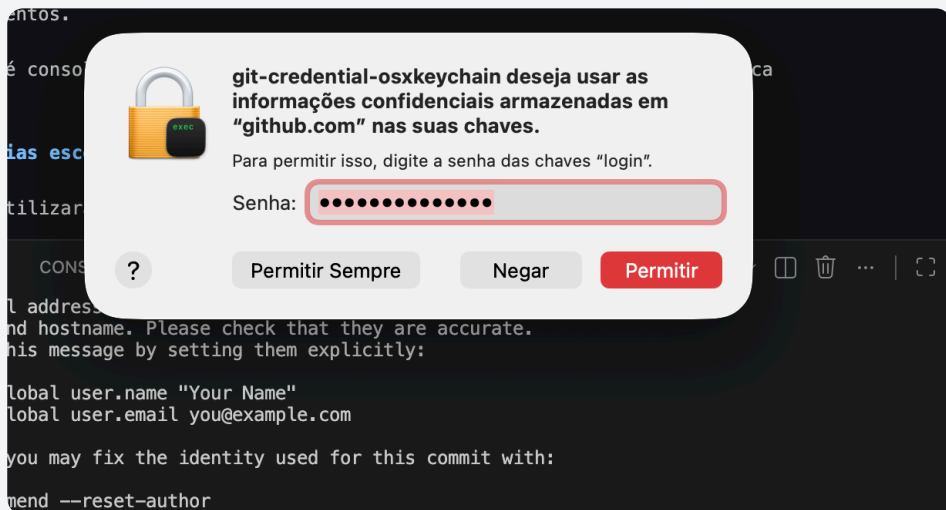
Obrigado
Galapagos Capital

 case-tecnico-estagio-painel-do-cliente.md

deu isso , me de um passo a passo para resolver isso



deu isso e eu nao sei a senha ... e agora



Isso é normal no Mac e **não é a senha do GitHub**. Essa janela está pedindo a senha do **Chaves do macOS (Keychain)**.

A senha normalmente é **a mesma senha que você usa para entrar/desbloquear o seu Mac**. Não me envie essa senha.

Faça assim agora

1. Apague o que você colocou no campo **Senha**.
2. Digite a senha que você usa quando o Mac pede sua senha para instalar programas ou alterar configurações.
3. Clique em **Permitir Sempre**.

Use **Permitir Sempre** porque, assim, o Git poderá acessar suas credenciais do GitHub sem perguntar toda hora.

Se funcionar, essa janela fecha e o terminal continua o comando que estávamos executando.

Se você realmente não souber a senha do Mac

Não fique tentando várias senhas. Clique em:

Negar

Não vamos estragar nada. O projeto e o commit continuam salvos.

Depois me mande **um print do que aparecer no terminal após clicar em “Negar”**. Aí eu te ensino outra forma de conectar ao GitHub usando o navegador, passo a passo.

E fique tranquilo: **até aqui seu projeto está certo**. Essa janela só apareceu porque o Git do seu Mac tentou acessar uma credencial salva no Chaves.



Você está **bem perto de terminar essa etapa**. Pela sua tela, quase tudo deu certo:

- ✓ Git configurado
- ✓ Nome configurado
- ✓ E-mail configurado
- ✓ Primeiro commit corrigido
- ✓ Repositório GitHub conectado como origin
- ✓ Branch mudou para main
- ✓ working tree clean
- ✗ Só falta autenticar no GitHub

O erro está exatamente aqui:

```
remote: Invalid username or token.  
Password authentication is not supported for Git operations.  
fatal: Authentication failed
```

Ou seja: **seus arquivos estão seguros**. O problema é somente o login do GitHub. O GitHub não aceita mais a senha normal da conta para `git push` via HTTPS; é necessário usar um método moderno de autenticação. **Authentication to GitHub - Git**

Vamos fazer da maneira mais fácil: **login pelo navegador usando GitHub CLI**, para você não precisar criar token manualmente. O próprio GitHub recomenda GitHub CLI ou Git Credential Manager para autenticação HTTPS. **GitHub credentials in Git -**

PASSO 1 — Não mexa em mais nada do projeto

Você está exatamente aqui:

```
bernardosilva@bernardo case-painel-cliente %
```

Continue nesse terminal.

Não execute `git init` novamente.

Não faça outro commit.

Não altere os arquivos.

PASSO 2 — Descobrir se o GitHub CLI já está instalado

Digite:

```
gh --version
```

e aperte **Enter**.

Se aparecer algo parecido com:

```
gh version 2.xx.x
```

Perfeito.

Vá direto para o **PASSO 3** abaixo.

Se aparecer:

```
command not found: gh
```

Pare aí e me mande um print.

Não precisa tentar instalar sozinho. Eu te passo o próximo comando certinho.

PASSO 3 — Fazer login no GitHub

Se o `gh --version` funcionou, digite:

```
gh auth login
```

Enter.

O GitHub CLI vai começar a fazer perguntas. Esse é justamente o processo oficial de autenticação pelo terminal. [da CLI GitHub - Documento](#)

Vai aparecer algo parecido com:

? Where do you use GitHub?

Escolha:

GitHub.com

Use as setinhas do teclado:

↑ ↓

e depois:

Enter

PASSO 4 — Escolher HTTPS

Depois provavelmente aparecerá:

What is your preferred protocol for Git operations?

As opções serão parecidas com:

HTTPS
SSH

Escolha:

HTTPS

Enter.

Isso combina com seu repositório atual, que já está configurado como:

`https://github.com/Mackenzie-TOTVS/case-painel-cliente.git`

PASSO 5 — Permitir que o Git use o login

Pode aparecer algo como:

Authenticate Git with your GitHub credentials?

Escolha:

Yes

ou:

Y

Isso é importante porque queremos que não apenas o comando `gh` tenha acesso ao GitHub, mas também comandos como:

`git push`

O GitHub CLI pode configurar e armazenar as credenciais usadas pelo Git para operações HTTPS.

r GitHub credentials in Git -

PASSO 6 — Escolher login pelo navegador

Provavelmente aparecerá:

How would you like to authenticate GitHub CLI?

Escolha:

Login with a web browser

Não escolha token manual.

Queremos a opção mais fácil.

PASSO 7 — Vai aparecer um código

O terminal provavelmente mostrará algo semelhante a:

First copy your one-time code:

ABCD-1234

Press Enter to open github.com in your browser...

Esse código será diferente.

Exemplo:

ABCD-1234

Não precisa me mandar esse código.

Copie o código.

Depois pressione:

Enter

PASSO 8 — O navegador será aberto

Seu navegador deverá abrir uma página do GitHub.

Faça login **na conta que tem acesso a este repositório:**

Mackenzie-TOTVS/case-painel-cliente

Depois o GitHub poderá pedir:

Device Activation

Digite ou cole o código mostrado no terminal.

Depois clique em algo parecido com:

Continue

e posteriormente:

Authorize GitHub

Pode ser solicitado 2FA caso esteja ativado na sua conta.

PASSO 9 — Voltar para o VS Code

Depois da autorização, volte para o terminal do VS Code.

Queremos que apareça algo parecido com:

- ✓ Authentication complete.
- ✓ Logged in as ...

Se isso aparecer:

✓ LOGIN FEITO

PASSO 10 — Conferir a conta conectada

Digite:

```
gh auth status
```

Enter.

Queremos algo semelhante a:

```
github.com
✓ Logged in to github.com account ...
✓ Git operations for github.com configured to use https protocol.
```

Se aparecer isso, perfeito.

PASSO 11 — Agora tente o `push` novamente

Digite:

```
git push -u origin main
```

Enter.

Desta vez queremos ver algo parecido com:

```
Enumerating objects...
Counting objects...
Writing objects...
To https://github.com/Mackenzie-TOTVS/case-painel-cliente.git

* [new branch] main -> main
branch 'main' set up to track 'origin/main'
```

Aí teremos:

SEU MAC

|
| git push



GITHUB



PASSO 12 — Conferir

Depois execute:

```
git status
```

Queremos:

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

Isso significa:

- ✓ Arquivos salvos
- ✓ Commit salvo
- ✓ GitHub conectado
- ✓ Projeto enviado

PASSO 13 — Atualizar seu GitHub

Volte para aquela página do GitHub que estava vazia.

Pressione:

```
Command + R
```

Agora deve aparecer:

```
case-painel-cliente  
|
```

```
| 01-INCEPTION.md
| AGENTS.md
```

E provavelmente a mensagem:

```
docs: registra concepcao inicial e skill do projeto
```

Aí completamos oficialmente nossa primeira parte.

Portanto, faça agora APENAS isso

Primeiro:

```
gh --version
```

Se funcionar, continue:

```
gh auth login
```

Escolha:

```
GitHub.com
↓
HTTPS
↓
Yes
↓
Login with a web browser
```

Depois:

```
gh auth status
```

E finalmente:

```
git push -u origin main
```

Se o primeiro comando `gh --version` mostrar `command not found`, pare exatamente aí e me mande o print. Não precisa tentar mais nada. Eu vou te ensinar a instalar a ferramenta e seguimos daqui, sem refazer nenhuma etapa. ☺memcite☺



deu isso

```
bernardosilva ~ — zsh — 80x24
Last login: Tue Aug 18 13:10:47 on console
bernardosilva@bernardo ~ % gh --version
zsh: command not found: gh
bernardosilva@bernardo ~ %
```



Perfeito — agora sabemos exatamente o que aconteceu: o comando `gh` **não está instalado no seu Mac**. Nada do seu projeto foi perdido e não precisamos refazer nenhum commit.

A forma mais simples no macOS é instalar o GitHub CLI pelo Homebrew; a própria página oficial do GitHub CLI indica `brew install gh` para macOS. [LI | Take GitHub to the comr](#)

Faça **somente estes passos agora**:

1. No terminal onde você está, digite:

```
brew --version
```

e pressione **Enter**.

Se aparecer algo parecido com:

```
Homebrew 4.x.x
```

então o Homebrew já está instalado. Nesse caso, digite:

```
brew install gh
```

Espre terminar. Pode demorar alguns minutos e aparecer bastante texto; isso é normal.

Quando terminar, confira:

```
gh --version
```

Se aparecer algo como:

```
gh version 2.x.x
```

a instalação deu certo.

Depois faça:

```
gh auth login
```

O próprio manual oficial do GitHub CLI indica `gh auth login` para autenticar a conta.

Manual | GitHub CLI

Quando ele fizer as perguntas, escolha nesta ordem:

```
GitHub.com
```

depois:

```
HTTPS
```

depois, se perguntar se quer autenticar o Git:

```
Yes
```

e depois:

```
Login with a web browser
```

Ele mostrará um código parecido com:

```
ABCD-1234
```

Não me envie esse código. Copie-o, pressione Enter e o navegador deverá abrir. Entre na conta do GitHub que possui acesso ao repositório `Mackenzie-TOTVS/case-painel-cliente`, cole o código e autorize.

Quando voltar ao terminal, faça:

```
gh auth status
```

Se aparecer que você está logado, aí volte para a pasta do projeto. Como nesse print você está em:

```
~
```

precisamos entrar novamente na pasta. Digite:

```
cd ~/Documents/case-painel-cliente
```

Depois confira:

```
git status
```

Deve aparecer:

```
On branch main
nothing to commit, working tree clean
```

Finalmente:

```
git push -u origin main
```

Aí o projeto deverá subir para o GitHub.

Mas atenção: faça primeiro apenas:

```
brew --version
```

Se aparecer `command not found: brew`, pare aí e me mande o print. Nesse caso eu te ensino a instalar o Homebrew primeiro, sem bagunçar nada do case. [memcite](#)



Sim, **deu certo**. Pela sua tela, o GitHub CLI confirmou:

```
✓ Authentication complete.
✓ Logged in as Mackenzie-TOTVS
```

E o `gh auth status` também confirmou:

```
✓ Logged in to github.com account Mackenzie-TOTVS
- Active account: true
- Git operations protocol: https
```

Então a autenticação está resolvida. Agora vamos continuar **exatamente de onde você parou**, sem pular para o código ainda.

ETAPA 2 — Enviar o primeiro commit para o GitHub

1. Na sua tela atual, você está nesta pasta:

```
~
```

O `~` significa sua pasta pessoal. Precisamos voltar para a pasta do projeto. Digite:

```
cd ~/Documents/case-painel-cliente
```

Aperte **Enter**.

O terminal deverá ficar parecido com:

```
bernardosilva@bernardo case-painel-cliente %
```

Agora confira:

```
git status
```

O esperado é algo assim:

```
On branch main
nothing to commit, working tree clean
```

Se aparecer isso, está perfeito.

Agora finalmente envie o primeiro commit:

```
git push -u origin main
```

Aperte **Enter**.

Se funcionar, aparecerá algo parecido com:

```
Enumerating objects...
Counting objects...
Writing objects...

To https://github.com/Mackenzie-TOTVS/case-painel-cliente.git
```

```
* [new branch] main -> main  
branch 'main' set up to track 'origin/main'
```

Depois execute novamente:

```
git status
```

Agora queremos:

```
On branch main  
  
Your branch is up to date with 'origin/main'.  
  
nothing to commit, working tree clean
```

Isso significa:

```
SEU MAC  
  |  
  | git push  
  ▼  
GITHUB  
  
✓ 01-INCEPTION.md  
✓ AGENTS.md  
✓ primeiro commit
```

Volte para o navegador onde está o GitHub e pressione:

```
Command + R
```

Você deve encontrar:

```
case-painel-cliente  
|  
├─ 01-INCEPTION.md  
└─ AGENTS.md
```

ETAPA 3 — Criar o planejamento antes do código

Agora vem uma etapa **muito importante do case**.

Eles exigem um arquivo chamado:

02-PLAN.md

e pedem que a solução seja quebrada em unidades de trabalho **antes da implementação**. Também querem que registremos ao longo do projeto quando uma proposta da IA for corrigida ou rejeitada.

Portanto ainda **não vamos criar** `index.html` .

No VS Code, na lateral esquerda, clique no botão de criar arquivo e digite:

02-PLAN.md

Sua estrutura ficará:

```
CASE-PAINEL-CLIENTE
|
├─ 01-INCEPTION.md
├─ 02-PLAN.md
└─ AGENTS.md
```

Abra o `02-PLAN.md` e coloque exatamente isto:

```
# 02 - Plano de Construção

## Objetivo

Construir um protótipo web simples de um Painel do Cliente utilizando HTML, CSS,
JavaScript e dados fictícios armazenados em JSON.

A implementação será realizada em pequenas etapas para que a evolução do projeto
possa ser acompanhada através do histórico de commits.

---

## Unidade 1 - Estruturar os dados fictícios

Criar um arquivo JSON representando um cliente fictício.

O arquivo deverá possuir:

- nome;
- segmento;
- telefone;
- e-mail;
- patrimônio em pelo menos três instituições;
- investimentos separados por categoria;
```

- histórico das últimas interações.

Resultado esperado:

Ter uma única fonte de dados fictícios para alimentar o painel.

Unidade 2 - Criar as regras de negócio

Criar funções JavaScript responsáveis pelos cálculos do painel.

Inicialmente serão necessárias funções para:

- calcular o patrimônio total;
- calcular o valor investido em cada categoria;
- calcular o percentual de cada categoria;
- identificar uma oportunidade de cross-sell.

As regras de negócio deverão ficar separadas da renderização da página.

Resultado esperado:

Os valores importantes do painel serão calculados a partir dos dados, evitando valores fixos diretamente no HTML.

Unidade 3 - Criar a estrutura visual do painel

Criar o arquivo HTML contendo as principais áreas:

- cabeçalho;
- informações do cliente;
- patrimônio consolidado;
- instituições;
- distribuição da alocação;
- histórico de relacionamento;
- oportunidade de cross-sell.

Resultado esperado:

Ter a estrutura completa da página, ainda sem preocupação principal com aparência.

Unidade 4 - Criar o estilo visual

Criar o CSS da aplicação.

O painel deverá possuir:

- organização em cards;
- boa legibilidade;
- destaque para o patrimônio total;
- representação visual da alocação;
- destaque para a oportunidade de cross-sell;
- adaptação básica para diferentes tamanhos de tela.

Resultado esperado:

Uma interface simples, organizada e adequada para demonstração.

Unidade 5 - Integrar os dados com a interface

Criar o JavaScript responsável por:

- carregar os dados fictícios;
- utilizar as funções de cálculo;
- apresentar os resultados no HTML;
- formatar valores monetários;
- apresentar as interações do cliente;
- apresentar a oportunidade identificada.

Resultado esperado:

Painel funcionando com informações geradas a partir dos dados fictícios.

Unidade 6 - Implementar a oportunidade de cross-sell

Criar uma regra simples e explicável.

A primeira ideia será analisar a distribuição patrimonial do cliente e identificar uma categoria com baixa participação na carteira.

A recomendação deverá informar:

- qual oportunidade foi encontrada;
- por que ela foi encontrada;
- qual tipo de produto o Advisor poderia conversar com o cliente.

A regra poderá ser ajustada durante o desenvolvimento caso produza uma recomendação pouco coerente.

Unidade 7 - Tratamento de erros

Adicionar tratamento para situações como:

- arquivo de dados não encontrado;
- estrutura de dados inválida;
- valores patrimoniais inválidos;
- campos essenciais ausentes.

O usuário deverá receber uma mensagem compreensível caso não seja possível carregar o painel.

Unidade 8 - Testes

Criar testes para as principais regras de negócio, especialmente:

- cálculo do patrimônio total;
- cálculo da alocação;
- cálculo de percentuais;
- regra de cross-sell;
- tratamento de valores inválidos.

Unidade 9 - Documentação final

Ao final do desenvolvimento:

- criar o 03-OPERATIONS.md;
- atualizar o AGENTS.md com a estrutura real do projeto;
- criar o README.md;
- exportar o histórico das conversas com IA;
- revisar todos os requisitos do case.

Estratégia de commits

Pretendo realizar commits incrementais.

Exemplo de evolução:

1. concepção e skill inicial;
2. planejamento da implementação;
3. criação dos dados fictícios;

4. regras de consolidação patrimonial;
5. estrutura inicial da interface;
6. estilização do painel;
7. integração dos dados;
8. oportunidade de cross-sell;
9. testes;
10. documentação e refinamentos.

Os commits deverão representar mudanças reais do projeto e não serão agrupados artificialmente ao final.

Registro de propostas da IA corrigidas ou rejeitadas

Esta seção será atualizada durante o desenvolvimento.

Até o momento, ainda não houve uma proposta técnica de implementação rejeitada após o início da construção.

Sempre que uma sugestão da IA for considerada inadequada, registrarei:

- o que a IA sugeriu;
- qual problema identifiquei;
- qual decisão tomei;
- por que escolhi a alternativa.

Salve:

Command + S

Por que estamos fazendo isso?

Porque agora nosso histórico mostra claramente:

PRIMEIRO

↓

Eu entendi o problema

01-INCEPTION.md

DEPOIS

↓

Eu planejei como resolver

02-PLAN.md

SÓ DEPOIS

↓

Comecei a programar

Isso conversa diretamente com o critério de avaliação do case sobre **intenção antes do código, commits incrementais e rastreabilidade das decisões**.

ETAPA 4 — Fazer o segundo commit

Agora abra o terminal dentro do VS Code.

Execute:

```
git status
```

Deve aparecer o novo arquivo:

```
02-PLAN.md
```

Agora:

```
git add 02-PLAN.md
```

Depois:

```
git commit -m "docs: adiciona plano de implementacao"
```

Depois:

```
git push
```

Agora nosso GitHub terá:

```
COMMIT 1  
docs: registra concepcao inicial e skill do projeto
```

↓

```
COMMIT 2  
docs: adiciona plano de implementacao
```

Isso é **muito melhor para esse case** do que criar tudo de uma vez.

Depois disso, pare

Quando terminar, sua pasta deve estar assim:

```
case-painel-cliente/  
|  
├─ 01-INCEPTION.md  
├─ 02-PLAN.md  
└─ AGENTS.md
```

E o GitHub deverá mostrar **2 commits**.

Não crie HTML, CSS, JavaScript nem JSON ainda.

O próximo passo será nossa **primeira etapa de programação de verdade**:

```
ETAPA 4  
CRIAR OS DADOS DO CLIENTE  
  
data/  
└─ client.json
```

E nessa etapa eu vou te ensinar o JSON desde o começo, inclusive o que significa cada `{ }`, `[ ]`, chave e valor. Depois faremos um commit **somente do JSON**, preservando exatamente a evolução que a Galapagos quer analisar.

Faça primeiro o `git push` do primeiro commit e depois o `02-PLAN.md`. Se qualquer comando der uma mensagem diferente, **pare naquele comando e me mande o print**; não precisa tentar corrigir sozinho.



Sim, **deu certinho**. Pela sua tela, o GitHub já mostra:

- ✓ 01-INCEPTION.md
- ✓ 02-PLAN.md
- ✓ AGENTS.md
- ✓ 2 commits
- ✓ branch main
- ✓ repositório privado

Estamos exatamente onde deveríamos estar. Agora vamos para a **primeira etapa de programação**: criar os dados fictícios do cliente em JSON. O case permite inventar os dados e inclusive sugere usar JSON para simular as diferentes instituições.

ETAPA 4 — Criar o arquivo `client.json`

PASSO 1 — Volte para o VS Code

Abra o projeto:

CASE-PAINEL-CLIENTE

Você deve ver:

```
CASE-PAINEL-CLIENTE
|
├── 01-INCEPTION.md
├── 02-PLAN.md
└── AGENTS.md
```

Agora vamos criar uma **pasta** chamada:

data

No lado esquerdo do VS Code, passe o mouse sobre:

CASE-PAINEL-CLIENTE

Clique no ícone de:

New Folder

Digite:

data

Pressione **Enter**.

Agora ficará:

```
CASE-PAINEL-CLIENTE
|
├── data/
├── 01-INCEPTION.md
├── 02-PLAN.md
└── AGENTS.md
```

PASSO 2 — Criar o arquivo dentro da pasta `data`

Clique com o botão direito em:

```
data
```

Depois:

```
New File
```

Digite exatamente:

```
client.json
```

Enter.

Agora:

```
CASE-PAINEL-CLIENTE
|
├─ data/
|   └─ client.json
|
├─ 01-INCEPTION.md
├─ 02-PLAN.md
└─ AGENTS.md
```

PASSO 3 — Copiar os dados do cliente

Abra:

```
client.json
```

Cole exatamente isto:

```
{
  "cliente": {
    "id": "CLI-001",
    "nome": "Mariana Costa",
    "segmento": "Private",
    "email": "mariana.costa@email.com",
    "telefone": "(11) 99999-1234"
  },
}
```

```

"instituicoes": [
  {
    "nome": "Itau",
    "posicoes": [
      {
        "produto": "CDB",
        "categoria": "Renda Fixa",
        "valor": 420000
      },
      {
        "produto": "Acoes",
        "categoria": "Renda Variavel",
        "valor": 180000
      },
      {
        "produto": "Fundo de Credito",
        "categoria": "Fundos",
        "valor": 200000
      }
    ]
  },
  {
    "nome": "BTG Pactual",
    "posicoes": [
      {
        "produto": "Tesouro Direto",
        "categoria": "Renda Fixa",
        "valor": 350000
      },
      {
        "produto": "Fundo Multimercado",
        "categoria": "Fundos",
        "valor": 400000
      },
      {
        "produto": "Dolar",
        "categoria": "Cambio",
        "valor": 200000
      }
    ]
  },
  {
    "nome": "XP Investimentos",
    "posicoes": [
      {
        "produto": "LCI",
        "categoria": "Renda Fixa",

```

```

        "valor": 300000
      },
      {
        "produto": "Acoes Brasileiras",
        "categoria": "Renda Variavel",
        "valor": 200000
      },
      {
        "produto": "Fundo Imobiliario",
        "categoria": "Fundos",
        "valor": 100000
      }
    ]
  }
],
"interacoes": [
  {
    "data": "2026-08-12",
    "canal": "Reuniao",
    "descricao": "Revisao da carteira e objetivos financeiros."
  },
  {
    "data": "2026-07-25",
    "canal": "Telefone",
    "descricao": "Cliente demonstrou interesse em diversificacao internacional."
  },
  {
    "data": "2026-07-10",
    "canal": "Email",
    "descricao": "Envio do relatorio mensal de investimentos."
  },
  {
    "data": "2026-06-18",
    "canal": "Reuniao",
    "descricao": "Discussao sobre perfil de risco e horizonte de investimento."
  }
]
}

```

Agora salve:

Command + S

PASSO 4 — Entenda o que acabamos de criar

Não precisa decorar JSON agora. Só quero que você entenda a ideia.

O começo:

```
"cliente": {
```

significa:

Aqui começam as informações do cliente.

Depois:

```
"nome": "Mariana Costa"
```

significa:

campo		valor
↓		↓
nome	→	Mariana Costa

Então:

```
"segmento": "Private"
```

é:

```
segmento → Private
```

Quando temos:

```
"instituicoes": [
```

o símbolo:

```
[  
]
```

significa:

teremos uma lista de instituições.

Dentro dela temos:

```
Instituição 1 → Itaú  
Instituição 2 → BTG Pactual
```

Isso atende diretamente à exigência de patrimônio vindo de **pelo menos três fontes/instituições**.

PASSO 5 — Veja os valores

Temos:

ITAÚ

CDB	R\$ 420.000
Ações	R\$ 180.000
Fundo	R\$ 200.000

Total Itaú	R\$ 800.000

BTG:

BTG PACTUAL

Tesouro	R\$ 350.000
Fundo	R\$ 400.000
Dólar	R\$ 200.000

Total BTG	R\$ 950.000

XP:

XP INVESTIMENTOS

LCI	R\$ 300.000
Ações	R\$ 200.000
Fundo	R\$ 100.000

Total XP	R\$ 600.000

Depois nosso JavaScript vai calcular:

```
R$ 800.000
+
R$ 950.000
+
R$ 600.000
```

R\$ 2.350.000

Muito importante: não colocamos:

```
"patrimonioTotal": 2350000
```

Por quê?

Porque queremos que **o programa faça a conta.**

Assim, se amanhã mudarmos:

```
"valor": 420000
```

para:

```
"valor": 500000
```

o sistema recalcula automaticamente.

Isso é melhor para demonstrar sua lógica.

PASSO 6 — Também preparamos nossa futura oportunidade

Repare que existe:

```
{  
  "produto": "Dolar",  
  "categoria": "Cambio",  
  "valor": 200000  
}
```

O patrimônio total será:

R\$ 2.350.000

E apenas:

R\$ 200.000

estão em câmbio.

Mais para frente poderemos criar uma regra parecida com:

```
SE exposição internacional/câmbio < 10%
    ↓
sinalizar oportunidade
    ↓
"Diversificação internacional"
```

Não vamos fazer isso agora.

Só estamos preparando bons dados para o sistema.

PASSO 7 — Conferir se o JSON está correto

Abra o terminal do VS Code:

```
Terminal
↓
New Terminal
```

Confira se aparece:

```
case-painel-cliente %
```

Digite:

```
python3 -m json.tool data/client.json
```

Enter.

Se estiver correto

O terminal simplesmente mostrará novamente todo o JSON organizado.

Não aparecerá erro.

Isso significa:

```
✓ JSON válido
```

Se aparecer algo como:

```
Expecting ',' delimiter
```

ou:

```
JSONDecodeError
```

pare e me mande o print.

Isso normalmente significa que alguma vírgula ou chave ficou faltando ao copiar.

PASSO 8 — Conferir com o Git

Agora:

```
git status
```

Deve aparecer algo parecido com:

```
Untracked files:
```

```
data/
```

Isso é normal.

O Git está dizendo:

Existe uma coisa nova que ainda não coloquei em um commit.

PASSO 9 — Adicionar o JSON

Digite:

```
git add data/client.json
```

Enter.

Depois confira:

```
git status
```

Agora deverá aparecer:

```
Changes to be committed:
```

```
new file: data/client.json
```

Perfeito.

PASSO 10 — Criar o terceiro commit

Digite:

```
git commit -m "feat: adiciona dados ficticios do cliente"
```

Enter.

Deve aparecer algo parecido com:

```
[main xxxxxxx] feat: adiciona dados ficticios do cliente  
  
1 file changed...  
create mode 100644 data/client.json
```

Agora nosso histórico é:

```
COMMIT 1  
docs: registra concepcao inicial e skill do projeto  
  
↓  
  
COMMIT 2  
docs: adiciona plano de implementacao  
  
↓  
  
COMMIT 3  
feat: adiciona dados ficticios do cliente
```

Isso é exatamente o tipo de evolução incremental que o case pede.

PASSO 11 — Subir para o GitHub

Agora:

```
git push
```

Enter.

Como já configuramos tudo antes, **não precisa mais usar:**

```
git push -u origin main
```

Agora normalmente basta:

```
git push
```

PASSO 12 — Conferir

Digite:

```
git status
```

Queremos:

```
On branch main
```

```
Your branch is up to date with 'origin/main'.
```

```
nothing to commit, working tree clean
```

Perfeito.

PASSO 13 — Conferir no GitHub

Volte ao navegador.

Atualize:

```
Command + R
```

Agora deverá aparecer:

```
case-painel-cliente
|
├─ data
|
├─ 01-INCEPTION.md
├─ 02-PLAN.md
└─ AGENTS.md
```

Clique em:

```
data
```

E deverá existir:

```
client.json
```

No topo do GitHub deverá mostrar:

3 Commits

Pare exatamente aqui

Por enquanto queremos:

- ✓ Conceção
- ✓ Planejamento
- ✓ Skill inicial
- ✓ Dados fictícios
- ✓ 3 instituições
- ✓ Histórico de relacionamento
- ✓ Terceiro commit
- ✓ Push para GitHub

Ainda não crie `index.html`.

Quando terminar essa etapa, me mande **um print do VS Code ou do GitHub mostrando `data/client.json` e os 3 commits.**

A próxima etapa será muito importante, mas vamos continuar do mesmo jeito, devagar:

ETAPA 5

```
src/  
└─ domain.js
```

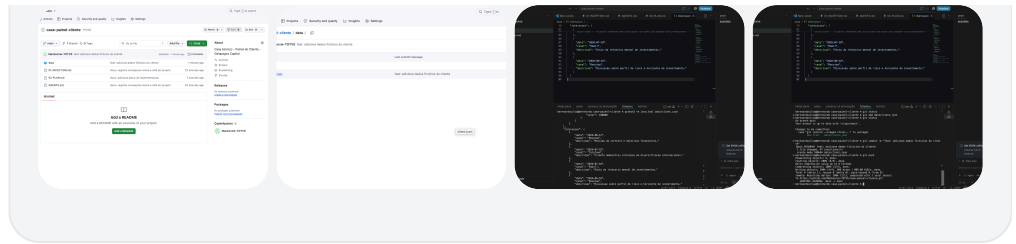
Primeira função:
calcular patrimônio total

E eu vou te explicar **linha por linha** o JavaScript que vamos escrever, inclusive o que cada `function`, `const`, `for`, parâmetro e `return` fazem antes de você subir o próximo commit.

☐memcite☐



acho que deu tudo certo , vamos para proxima etapa , me de um passo a passo desde criar o arquivo ate a etapa final



Sim. **Deu tudo certo.** Pelos seus prints, o `client.json` está válido, o terceiro commit foi criado e o `git push` chegou ao GitHub. Você está com **3 commits**, exatamente como planejamos. ✅

Agora vamos para a próxima etapa: **criar as regras de negócio do painel**. Ainda não vamos fazer a tela. Primeiro vamos ensinar o JavaScript a fazer as contas, porque o case também avalia organização, legibilidade, tratamento de entrada inválida e domínio da solução.

ETAPA 5 — Criar o `domain.js`

No final desta etapa teremos:

```
case-painel-cliente/
|
├── data/
│   └── client.json
|
├── src/
│   └── domain.js      ← NOVO
|
├── 01-INCEPTION.md
├── 02-PLAN.md
└── AGENTS.md
```

Esse arquivo será o **cérebro das contas**.

PASSO 1 — Criar a pasta `src`

Volte ao VS Code.

No lado esquerdo, passe o mouse sobre:

CASE-PAINEL-CLIENTE

Clique em:

New Folder

Digite:

```
src
```

Pressione **Enter**.

Agora deverá ficar:

```
CASE-PAINEL-CLIENTE
```

```
data/  
  client.json
```

```
src/
```

```
01-INCEPTION.md  
02-PLAN.md  
AGENTS.md
```

PASSO 2 — Criar `domain.js`

Clique com o botão direito na pasta:

```
src
```

Depois:

```
New File
```

Nome:

```
domain.js
```

Então:

```
src/  
└─ domain.js
```

Abra esse arquivo.

PASSO 3 — Cole este código inteiro

Copie e cole **exatamente**:

```

function validarInstituicoes(instituicoes) {
  if (!Array.isArray(instituicoes) || instituicoes.length === 0) {
    throw new Error("A lista de instituicoes e obrigatoria.");
  }

  for (const instituicao of instituicoes) {
    if (!instituicao.nome || !Array.isArray(instituicao.posicoes)) {
      throw new Error("Instituicao com dados invalidos.");
    }

    for (const posicao of instituicao.posicoes) {
      if (
        !posicao.categoria ||
        !Number.isFinite(posicao.valor) ||
        posicao.valor < 0
      ) {
        throw new Error("Posicao patrimonial invalida.");
      }
    }
  }
}

export function calcularPatrimonioTotal(instituicoes) {
  validarInstituicoes(instituicoes);

  let patrimonioTotal = 0;

  for (const instituicao of instituicoes) {
    for (const posicao of instituicao.posicoes) {
      patrimonioTotal += posicao.valor;
    }
  }

  return patrimonioTotal;
}

export function calcularPatrimonioPorInstituicao(instituicoes) {
  validarInstituicoes(instituicoes);

  const resultado = [];

  for (const instituicao of instituicoes) {
    let totalInstituicao = 0;

    for (const posicao of instituicao.posicoes) {
      totalInstituicao += posicao.valor;
    }
  }
}

```

```

        resultado.push({
            nome: instituicao.nome,
            valor: totalInstituicao
        });
    }

    return resultado;
}

export function calcularAlocacao(instituicoes) {
    validarInstituicoes(instituicoes);

    const patrimonioTotal = calcularPatrimonioTotal(instituicoes);

    if (patrimonioTotal === 0) {
        throw new Error("O patrimonio total nao pode ser zero.");
    }

    const valoresPorCategoria = {};

    for (const instituicao of instituicoes) {
        for (const posicao of instituicao.posicoes) {
            const categoria = posicao.categoria;

            if (!valoresPorCategoria[categoria]) {
                valoresPorCategoria[categoria] = 0;
            }

            valoresPorCategoria[categoria] += posicao.valor;
        }
    }

    const alocao = [];

    for (const categoria in valoresPorCategoria) {
        const valor = valoresPorCategoria[categoria];

        const percentual = (valor / patrimonioTotal) * 100;

        alocao.push({
            categoria: categoria,
            valor: valor,
            percentual: Number(percentual.toFixed(2))
        });
    }

    return alocao;
}

```

```

export function identificarOportunidadeCrossSell(alocacao) {
  if (!Array.isArray(alocacao) || alocacao.length === 0) {
    throw new Error("A alocacao e obrigatoria.");
  }

  const cambio = alocacao.find(
    (item) => item.categoria === "Cambio"
  );

  if (!cambio || cambio.percentual < 10) {
    return {
      titulo: "Diversificacao internacional",
      produtoSugerido: "Produtos com exposicao internacional",
      justificativa:
        "O cliente possui menos de 10% do patrimonio na categoria Cambio. Existe oportunidade de conversar sobre maior diversificacao internacional."
    };
  }

  return {
    titulo: "Revisao de diversificacao",
    produtoSugerido: "Produto complementar a carteira",
    justificativa:
      "A carteira possui exposicao internacional relevante. O Advisor pode avaliar outras categorias com menor participacao."
  };
}

```

Agora salve:

Command + S

PASSO 4 — Entenda o que acabamos de criar

Parece muita coisa, mas na realidade temos **4 funções principais**.

Pense assim:

```

client.json
  ↓
  ↓
domain.js
  ↓
[ faz as contas ]

```

↓
resultado

Vamos entender uma por uma.

Função 1 — `calcularPatrimonioTotal`

Temos:

```
export function calcularPatrimonioTotal(instituicoes) {
```

Significa:

Criei uma função chamada `calcularPatrimonioTotal`.

Ela recebe:

`instituicoes`

Ou seja:

Itaú
BTG
XP

Depois:

```
let patrimonioTotal = 0;
```

Imagine uma caixinha:

`patrimonioTotal`

0

O programa começa em zero.

Depois ele passa pelas instituições e posições:

```
patrimonioTotal += posicao.valor;
```

Ou seja:

```
0
+ 420.000
= 420.000

420.000
+ 180.000
= 600.000

600.000
+ 200.000
= 800.000

...
```

Até chegar em:

R\$ 2.350.000

Finalmente:

```
return patrimonioTotal;
```

Significa:

devolva o resultado.

Portanto:

```
calcularPatrimonioTotal(...)
      ↓
    2350000
```

Função 2 — Patrimônio de cada instituição

Esta:

```
calcularPatrimonioPorInstituicao()
```

faz separadamente:

```
ITAÚ

420.000
+ 180.000
```

+ 200.000

800.000

Depois:

BTG

350.000

+ 400.000

+ 200.000

950.000

Depois:

XP

300.000

+ 200.000

+ 100.000

600.000

O resultado ficará parecido com:

```
[
  {
    nome: "Itau",
    valor: 800000
  },
  {
    nome: "BTG Pactual",
    valor: 950000
  },
  {
    nome: "XP Investimentos",
    valor: 600000
  }
]
```

Mais para frente vamos transformar isso na tela:

Patrimônio por instituição

Itaú	R\$ 800.000
BTG Pactual	R\$ 950.000
XP Investimentos	R\$ 600.000

Função 3 — calcularAlocacao

Agora precisamos descobrir quanto existe em cada categoria.

No nosso JSON temos:

Renda Fixa

Itaú CDB	420.000
BTG Tesouro	350.000
XP LCI	300.000
Total	1.070.000

Renda Variável

Itaú	180.000
XP	200.000
Total	380.000

Fundos

Itaú	200.000
BTG	400.000
XP	100.000
Total	700.000

Câmbio

BTG	200.000
-----	---------

Total geral:

2.350.000

O JavaScript calcula os percentuais:

Renda Fixa

1.070.000 / 2.350.000

≈ 45,53%

Renda Variável

380.000 / 2.350.000

≈ 16,17%

Fundos

700.000 / 2.350.000

≈ 29,79%

Câmbio

200.000 / 2.350.000

≈ 8,51%

Depois nosso painel mostrará algo parecido:



Isso atenderá ao requisito de **distribuição visual da alocação**.

Função 4 — Cross-sell

Agora temos:

```
identificarOportunidadeCrossSell()
```

A regra que escolhemos é:

Câmbio < 10%?

No nosso caso:

Câmbio = 8,51%

Então:

$8,51 < 10$

SIM

O sistema gera:

OPORTUNIDADE

Diversificação internacional

O cliente possui menos de 10% do patrimônio na categoria Câmbio.

Sugestão:

Produtos com exposição internacional.

Isso será colocado em destaque no painel.

E é importante porque o case exige uma oportunidade de cross-sell **com justificativa simples**.

E para que serve `validarInstituicoes()` ?

Essa é uma função de segurança.

Por exemplo, se por acidente aparecer:

```
"valor": -500000
```

ela identifica:

✗ Valor negativo

E gera:

Posicao patrimonial invalida.

Ou se não houver instituições:

❌ lista vazia

ela também gera erro.

Isso é bom porque o case menciona explicitamente **tratamento de entrada inválida** como parte da qualidade técnica avaliada.

PASSO 5 — Veja se o VS Code mostra erro

Observe o arquivo.

Na parte inferior esquerda do VS Code existe algo parecido com:

ⓧ 0 ⚠️ 0

Se estiver:

0 erros

ótimo.

Se houver sublinhados vermelhos no código, me mande um print antes de continuar.

PASSO 6 — Conferir no Git

Agora vamos registrar essa evolução.

Abra:

Terminal

Digite:

git status

Deve aparecer:

Untracked files:

src/

Isso significa simplesmente:

existe um arquivo novo.

PASSO 7 — Adicionar o `domain.js`

Digite:

```
git add src/domain.js
```

Enter.

Agora:

```
git status
```

Deve aparecer:

```
Changes to be committed:
  new file:   src/domain.js
```

Perfeito.

PASSO 8 — Criar nosso quarto commit

Digite:

```
git commit -m "feat: adiciona regras de calculo patrimonial"
```

Enter.

Você deverá ver algo parecido com:

```
[main xxxxxxx] feat: adiciona regras de calculo patrimonial
1 file changed...
create mode 100644 src/domain.js
```

Agora nosso histórico está ficando muito bom:

```
COMMIT 1
Concepção
    ↓
docs: registra concepcao inicial e skill do projeto

COMMIT 2
```

Planejamento

↓

docs: adiciona plano de implementacao

COMMIT 3

Dados

↓

feat: adiciona dados ficticios do cliente

COMMIT 4

Regras

↓

feat: adiciona regras de calculo patrimonial

Isso preserva a evolução que o case exige.

PASSO 9 — Enviar para o GitHub

Digite:

```
git push
```

Enter.

Espere aparecer algo semelhante a:

```
To https://github.com/...  
xxxxxxx..xxxxxxx  main -> main
```

PASSO 10 — Conferir novamente

Digite:

```
git status
```

Queremos:

```
On branch main  
  
Your branch is up to date with 'origin/main'.  
  
nothing to commit, working tree clean
```

Excelente.

PASSO 11 — Conferir no GitHub

Volte ao GitHub.

Atualize:

Command + R

Agora deve aparecer:

```
case-painel-cliente/  
|  
├─ data/  
|   └─ client.json  
|  
├─ src/  
|   └─ domain.js  
|  
├─ 01-INCEPTION.md  
├─ 02-PLAN.md  
└─ AGENTS.md
```

E no topo:

4 Commits

PASSO 12 — Clique em **src**

Entre:

src

E confira se aparece:

domain.js

Se sim:

✓ ETAPA 5 CONCLUÍDA

Onde estamos no projeto

Nosso progresso agora será:

- ✓ 01 – Concepção
- ✓ 02 – Planejamento
- ✓ 03 – Skill inicial
- ✓ 04 – Dados fictícios
- ✓ 05 – Regras de negócio

PRÓXIMO



06 – Criar o HTML

A próxima etapa será bem mais visual.

Vamos criar:

```
index.html
```

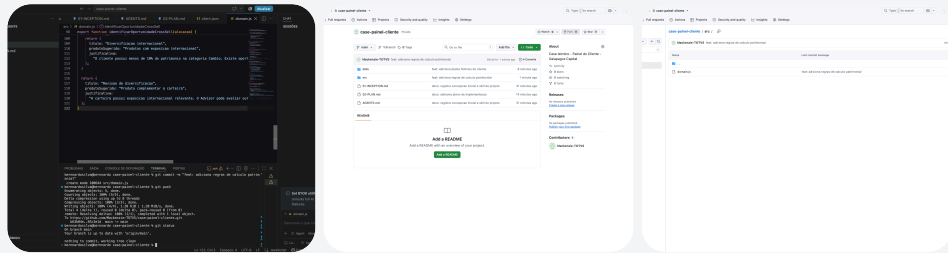
e começar a construir:

PAINEL DO CLIENTE
Mariana Costa
Private
Patrimônio total
R\$ 2.350.000
Itaú BTG XP
Alocação
Histórico
Oportunidade de Cross-sell

Pare depois do quarto commit e do `git push`. Não crie o HTML ainda. Quando aparecerem `src/domain.js` e **4 Commits** no GitHub, me mande o print e continuamos do zero com o `index.html`: criar arquivo → copiar código → explicar linha por linha → conferir → commit → push. [memcite](#)



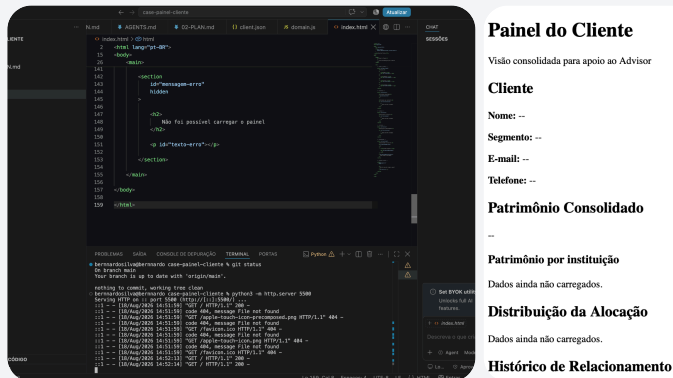
acho que deu tudo certo , me de um passo a passo da proxima etapa , bem passo a passo mesmo



me gere todo o passo a passo novamente , me de o codigo do index , mais nao de como editar , me todo o passo a passo novamente



acho que tem algo de errado ? ou esta certo ?



Está **certo**. Para esta etapa, sua página ficou exatamente como deveria.

Pelo seu print, temos:

- ✓ O servidor abriu corretamente
- ✓ O index.html foi encontrado
- ✓ Todos os títulos aparecem
- ✓ As seções aparecem
- ✓ Os "--" aparecem onde os dados entrarão depois
- ✓ "Dados ainda não carregados" aparece corretamente

No terminal também aparece:

```
"GET / HTTP/1.1" 200
```

O `200` significa:

✅ A página foi carregada com sucesso.

E aqueles `404` no terminal?

Você tem mensagens como:

```
GET /favicon.ico HTTP/1.1 404  
GET /apple-touch-icon.png HTTP/1.1 404
```

Isso não é problema do projeto.

O navegador simplesmente tentou procurar um ícone para a aba:

```
favicon.ico
```

e um ícone usado por dispositivos Apple:

```
apple-touch-icon.png
```

Como nós não criamos esses ícones, ele responde:

```
404 = arquivo não encontrado
```

Pode ignorar completamente por enquanto.

Por que está tão simples?

Porque neste momento nós temos somente:

```
HTML  
↓  
estrutura da página
```

Ainda não temos:

```
CSS  
↓
```

aparência

nem:

```
app.js
↓
colocar os dados na página
```

Por isso aparece:

```
Nome: --
Segmento: --
E-mail: --
Telefone: --
```

e:

Dados ainda não carregados.

Isso é **proposital**.

Mais para frente teremos:

```
client.json
↓
app.js
↓
domain.js
↓
index.html
```

E aí o que hoje está assim:

Nome: --

virará:

Nome: Mariana Costa

E:

Patrimônio Consolidado

```
--
```

virará:

```
Patrimônio Consolidado
```

```
R$ 2.350.000,00
```

Então **não altere o HTML agora.**

Pode fazer o commit

Primeiro volte ao terminal e encerre o servidor com:

```
Control + C
```

Depois:

```
git status
```

Depois:

```
git add index.html
```

Depois:

```
git commit -m "feat: adiciona estrutura inicial do painel"
```

Depois:

```
git push
```

E finalmente:

```
git status
```

Queremos:

```
nothing to commit, working tree clean
```

No GitHub deverá aparecer **5 Commits.**

Depois disso, a próxima etapa é o `styles.css`, que é quando finalmente vamos deixar esse painel com aparência profissional: cards, espaçamento, fundo, títulos, patrimônio destacado e preparação das barras de alocação.



Sim, **deu tudo certinho**. Pelos seus prints:

- ✓ index.html criado
- ✓ quinto commit criado
- ✓ git push realizado
- ✓ GitHub mostra 5 Commits
- ✓ git status mostra working tree clean

Estamos exatamente no ponto correto.

Agora vamos para a próxima etapa, **do mesmo jeitinho**: criar o arquivo, copiar o código, conectar, testar no navegador, fazer commit e subir no GitHub.

ETAPA 7 — DEIXAR O PAINEL BONITO COM CSS

Agora criaremos:

```
styles.css
```

Pense assim:

```
index.html
  ↓
estrutura

styles.css
  ↓
aparência

domain.js
  ↓
cálculos
```

Hoje sua página está assim:

```
Painel do Cliente
```

```
Cliente
```

```
Nome: --
```

```
Segmento: --
```

Patrimônio...

Depois do CSS queremos algo mais próximo de:

PAINEL DO CLIENTE Visão consolidada para apoio ao Advisor	
CLIENTE Mariana Costa	PATRIMÔNIO R\$ 2.350.000
DISTRIBUIÇÃO DA ALOCAÇÃO ████████████████████	

Ainda não aparecerão os dados reais. Isso será na próxima etapa.

PASSO 1 — Criar o arquivo `styles.css`

Volte ao VS Code.

Você está com:

```
CASE-PAINEL-CLIENTE
|
├─ data/
├─ src/
├─ 01-INCEPTION.md
├─ 02-PLAN.md
├─ AGENTS.md
└─ index.html
```

Passe o mouse sobre:

CASE-PAINEL-CLIENTE

Clique em:

New File

Digite:

```
styles.css
```

Pressione Enter.

Sua estrutura ficará:

```
CASE-PAINEL-CLIENTE
|
├─ data/
|   └─ client.json
|
├─ src/
|   └─ domain.js
|
├─ index.html
├─ styles.css      ← NOVO
├─ 01-INCEPTION.md
├─ 02-PLAN.md
└─ AGENTS.md
```

PASSO 2 — Abrir o `styles.css`

Clique em:

```
styles.css
```

O arquivo estará vazio.

Copie **todo** o código abaixo.

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, Helvetica, sans-serif;
  background-color: #f4f6f8;
  color: #1f2937;
}

header {
  background-color: #111827;
  color: white;
```

```

padding: 32px 40px;
}

header h1 {
margin: 0 0 8px;
font-size: 32px;
}

header p {
margin: 0;
color: #d1d5db;
}

main {
max-width: 1200px;
margin: 0 auto;
padding: 32px 20px;

display: grid;
grid-template-columns: repeat(2, 1fr);
gap: 24px;
}

section {
background-color: white;
border-radius: 12px;
padding: 24px;

border: 1px solid #e5e7eb;

box-shadow: 0 2px 8px rgba(0, 0, 0, 0.05);
}

section h2 {
margin-top: 0;
margin-bottom: 20px;
font-size: 20px;
color: #111827;
}

section h3 {
color: #374151;
}

#datos-cliente p {
margin: 12px 0;
}

#patrimonio-total {

```

```

    font-size: 34px;
    font-weight: bold;
    margin: 10px 0 28px;
    color: #11827;
}

#alocacao {
    grid-column: 1 / -1;
}

#historico {
    grid-column: 1 / -1;
}

#oportunidade {
    grid-column: 1 / -1;

    border-left: 5px solid #2563eb;
    background-color: #eff6ff;
}

#oportunidade h2 {
    color: #1d4ed8;
}

#oportunidade-titulo {
    font-size: 22px;
    margin-bottom: 12px;
}

#mensagem-erro {
    grid-column: 1 / -1;

    border-left: 5px solid #dc2626;
    background-color: #fef2f2;
}

.instituicao-card {
    border-bottom: 1px solid #e5e7eb;
    padding: 12px 0;
}

.instituicao-card:last-child {
    border-bottom: none;
}

.instituicao-nome {
    font-weight: bold;
}

```

```
.instituicao-valor {
  margin-top: 5px;
  color: #4b5563;
}

.alocacao-item {
  margin-bottom: 20px;
}

.alocacao-cabecalho {
  display: flex;
  justify-content: space-between;
  margin-bottom: 8px;
}

.barra {
  width: 100%;
  height: 12px;
  border-radius: 6px;
  background-color: #e5e7eb;
  overflow: hidden;
}

.barra-preenchimento {
  height: 100%;
  background-color: #2563eb;
  border-radius: 6px;
}

.interacao-item {
  padding: 16px 0;
  border-bottom: 1px solid #e5e7eb;
}

.interacao-item:last-child {
  border-bottom: none;
}

.interacao-data {
  font-size: 14px;
  color: #6b7280;
}

.interacao-canal {
  display: inline-block;

  margin-top: 6px;
  margin-bottom: 6px;
}
```

```
padding: 4px 8px;

border-radius: 6px;

background-color: #e5e7eb;

font-size: 13px;
font-weight: bold;
}

@media (max-width: 768px) {

  header {
    padding: 24px 20px;
  }

  header h1 {
    font-size: 26px;
  }

  main {
    grid-template-columns: 1fr;
  }

  #alocacao,
  #historico,
  #oportunidade,
  #mensagem-erro {
    grid-column: auto;
  }

  #patrimonio-total {
    font-size: 28px;
  }
}
```

Agora salve:

Command + S

PASSO 3 — O que esse CSS faz?

Não precisa decorar.

A ideia é simples.

Quando temos:

```
body {  
  background-color: #f4f6f8;  
}
```

estamos dizendo:

deixe o fundo da página cinza bem claro.

Quando temos:

```
section {  
  background-color: white;  
  border-radius: 12px;  
}
```

estamos dizendo:

transforme cada seção em um card branco com cantos arredondados.

Então:

```
<section>  
Cliente  
</section>
```

vai parecer um card.

PASSO 4 — Entenda o `main`

Temos:

```
main {  
  display: grid;  
  grid-template-columns: repeat(2, 1fr);  
}
```

Isso significa que no computador teremos:

COLUNA 1	COLUNA 2
Cliente	Patrimônio



Em vez de tudo ficar um embaixo do outro.

PASSO 5 — E no celular?

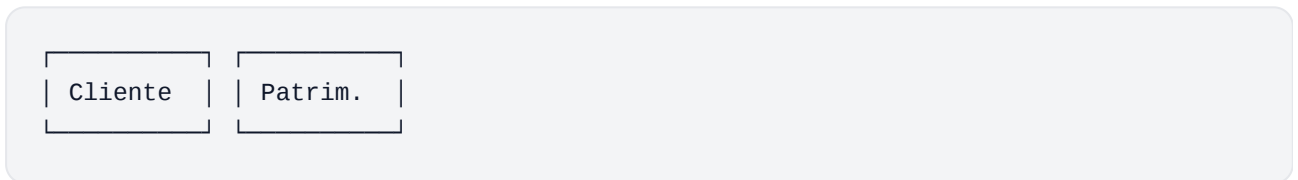
No final do CSS temos:

```
@media (max-width: 768px)
```

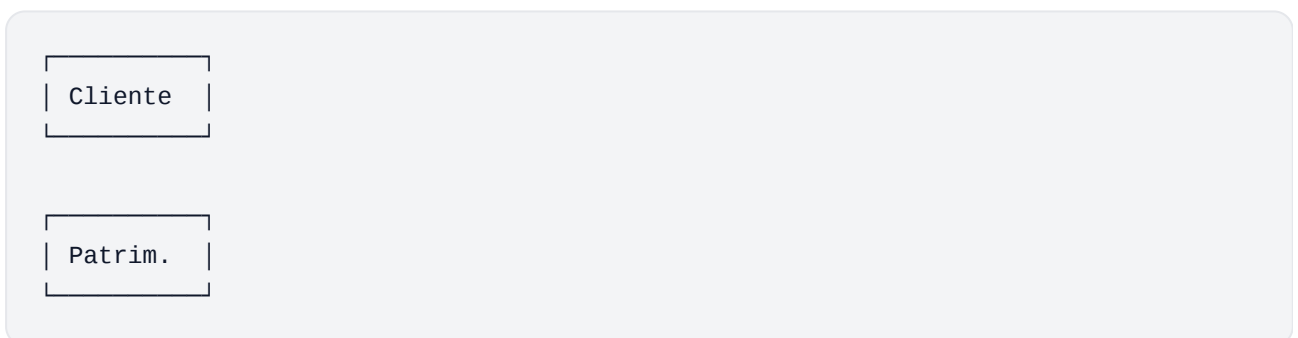
Isso significa:

se a tela ficar pequena, mude o layout.

No computador:



No celular:



Isso dá uma responsividade básica ao projeto.

PASSO 6 — Ainda falta uma coisa importante

Só criar:

```
styles.css
```

não é suficiente.

Precisamos avisar ao `index.html`:

utilize aquele arquivo CSS.

Abra:

```
index.html
```

Vá para o começo.

Você deverá encontrar:

```
<head>
  <meta charset="UTF-8">
```

e depois:

```
<title>Painel do Cliente</title>
```

Agora precisamos colocar uma linha **antes de** `</head>` .

Deixe essa parte assim:

```
<head>
  <meta charset="UTF-8">

  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >

  <title>Painel do Cliente</title>

  <link rel="stylesheet" href="styles.css">
</head>
```

A linha nova é:

```
<link rel="stylesheet" href="styles.css">
```

PASSO 7 — Entenda essa linha

Isso:

```
<link rel="stylesheet" href="styles.css">
```

significa:

```
index.html
  |
  | use
  ▼
styles.css
```

O:

```
href="styles.css"
```

indica onde está o arquivo.

Como os dois estão na mesma pasta:

```
case-painel-cliente/
|
├─ index.html
└─ styles.css
```

podemos simplesmente utilizar:

```
styles.css
```

Salve o `index.html` :

```
Command + S
```

PASSO 8 — Agora vamos testar

Abra o terminal.

Digite:

```
python3 -m http.server 5500
```

Pressione Enter.

Deve aparecer:

```
Serving HTTP on ... port 5500
```

PASSO 9 — Abra o navegador

No Safari ou Chrome:

```
http://localhost:5500
```

Se já estiver aberto, apenas atualize:

```
Command + R
```

PASSO 10 — O que deve mudar?

Antes você tinha:

```
Painel do Cliente
```

```
Cliente
```

```
Nome: --
```

```
Segmento: --
```

Agora deverá ver algo bem diferente.

O fundo deverá ficar cinza claro.

No topo deverá aparecer uma área escura:

```
| PAINEL DO CLIENTE |  
| Visão consolidada para apoio ao Advisor |
```

Depois:

CLIENTE	PATRIMÔNIO
Nome: --	--
Segmento: --	
Email: --	Dados ainda...
Telefone: --	

Depois teremos cards maiores:

DISTRIBUIÇÃO DA ALOCAÇÃO

Dados ainda não carregados.

E a oportunidade terá um destaque azul.

PASSO 11 — Não se preocupe com os dados vazios

Mesmo depois do CSS continuará aparecendo:

Nome: - -

e:

Dados ainda não carregados.

Isso continua correto.

Temos agora:

✓ HTML
estrutura

✓ CSS
aparência

✗ app.js
integração

É justamente o `app.js` da próxima etapa que vai preencher tudo.

PASSO 12 — Confira o celular rapidamente

No navegador, diminua a largura da janela.

Você deve perceber que os dois cards:

Cliente | Patrimônio

passam a ficar:

Cliente



Patrimônio

Se isso acontecer:

✓ responsividade funcionando

PASSO 13 — Parar o servidor

Volte ao terminal.

Pressione:

Control + C

Você voltará para:

bernardosilva@bernardo case-painel-cliente %

PASSO 14 — Conferir com Git

Digite:

git status

Agora deverá aparecer algo parecido com:

Changes not staged for commit:

modified: index.html

Untracked files:

styles.css

Isso está **perfeito**.

Por quê?

Porque:

```
index.html
```

foi modificado para conectar o CSS.

E:

```
styles.css
```

é um arquivo novo.

PASSO 15 — Adicionar os dois arquivos

Digite:

```
git add index.html styles.css
```

Enter.

Depois:

```
git status
```

Agora deverá aparecer algo semelhante a:

```
Changes to be committed:
```

```
    modified: index.html
```

```
    new file: styles.css
```

Perfeito.

PASSO 16 — Criar o sexto commit

Digite:

```
git commit -m "feat: adiciona estilo visual do painel"
```

Enter.

Queremos algo parecido com:

```
[main xxxxxxx] feat: adiciona estilo visual do painel
```

```
2 files changed...
create mode 100644 styles.css
```

Nosso histórico agora ficará:

```
COMMIT 1
Concepção

↓

COMMIT 2
Planejamento

↓

COMMIT 3
Dados fictícios

↓

COMMIT 4
Regras de negócio

↓

COMMIT 5
Estrutura HTML

↓

COMMIT 6
Estilo visual
```

Isso continua preservando a evolução incremental exigida pelo case.

PASSO 17 — Enviar ao GitHub

Digite:

```
git push
```

Enter.

Espere terminar.

Você provavelmente verá:

```
main -> main
```

PASSO 18 — Conferir se está tudo certo

Digite:

```
git status
```

Queremos:

```
On branch main
```

```
Your branch is up to date with 'origin/main'.
```

```
nothing to commit, working tree clean
```

A parte mais importante:

```
nothing to commit, working tree clean
```

significa:

- ✓ tudo salvo
- ✓ tudo commitado
- ✓ nada pendente

PASSO 19 — Conferir no GitHub

Volte para o GitHub.

Pressione:

```
Command + R
```

Agora esperamos encontrar:

```
case-painel-cliente/  
|  
├─ data/  
├─ src/  
├─ index.html  
└─ styles.css
```



```
├─ 01-INCEPTION.md
├─ 02-PLAN.md
└─ AGENTS.md
```

E no topo:

6 Commits

Se aparecer:

6 Commits

essa etapa terminou.

PARE AQUI

Ao terminar esta etapa teremos:

- ✓ 01-INCEPTION.md
- ✓ 02-PLAN.md
- ✓ AGENTS.md

- ✓ client.json
- ✓ domain.js
- ✓ index.html
- ✓ styles.css

- ✓ HTML funcionando
- ✓ CSS funcionando
- ✓ layout responsivo básico

- ✓ 6 commits
- ✓ tudo enviado ao GitHub

E então vem a etapa **mais importante até agora**.

PRÓXIMA ETAPA — `src/app.js`

É nela que finalmente conectaremos tudo:

```
client.json
  ↓
  app.js
  ↓
domain.js
```

↓
index.html

E finalmente:

Nome: --

vai virar:

Nome: Mariana Costa

O:

--

do patrimônio vai virar:

R\$ 2.350.000,00

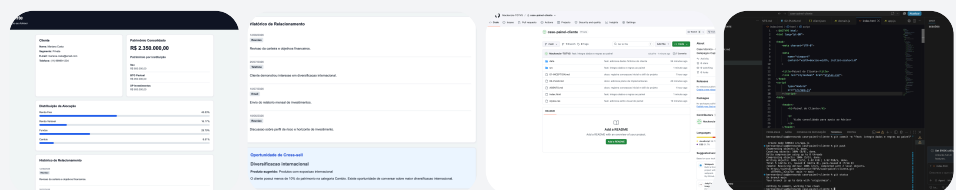
E aparecerão:

Itaú	R\$ 800.000
BTG Pactual	R\$ 950.000
XP Investimentos	R\$ 600.000

Além das barras de alocação, histórico e oportunidade de cross-sell.

Faça somente esta ETAPA 7 agora. Quando o navegador estiver estilizado e o GitHub mostrar **6 Commits**, me mande o print. Aí fazemos o `app.js` passo a passo, desde criar o arquivo até testar, fazer commit e subir.

deu tudo certo , qual a proxima etapa ? me direciona desde o zero , passando parte por parte , igual um tutorial me ajude ... me diga qual arquivo devo criar e oque colocar nele





Sim — **deu tudo certo**. Pelos seus prints, o painel já está funcionando com os dados reais e o GitHub mostra **7 commits**.

Você já cumpriu os cinco requisitos funcionais principais do painel: dados do cliente, patrimônio de 3 instituições, alocação visual, histórico e cross-sell.

Agora vamos para a próxima etapa, no mesmo formato que estamos usando:

ETAPA 9 — CRIAR TESTES AUTOMÁTICOS

Agora queremos provar que nossas contas realmente funcionam.

O próprio case diz que a presença de **testes** faz parte da avaliação técnica.

No final desta etapa teremos:

```
case-painel-cliente/
|
├─ data/
|   └─ client.json
|
├─ src/
|   ├── app.js
|   └─ domain.js
|
├─ tests/
|   └─ domain.test.js      ← NOVO
|
├─ package.json           ← NOVO
|
├─ index.html
├─ styles.css
├─ 01-INCEPTION.md
├─ 02-PLAN.md
└─ AGENTS.md
```

E poderemos executar:

```
npm test
```

e o computador vai verificar automaticamente nossas contas.

PASSO 1 — Primeiro verificar se você tem Node.js

Abra o terminal do VS Code.

Digite:

```
node --version
```

Pressione Enter.

Se aparecer algo parecido com:

```
v22.18.0
```

ou:

```
v20.19.0
```

ou qualquer versão moderna, perfeito.

Depois execute:

```
npm --version
```

Se aparecer um número, por exemplo:

```
10.9.3
```

também está certo.

Se aparecer:

```
command not found: node
```

Pare aí e me mande um print.

Nesse caso primeiro instalamos o Node.

Se `node --version` e `npm --version` funcionarem, continue.

PASSO 2 — Criar o `package.json`

No VS Code, passe o mouse sobre:

```
CASE-PAINEL-CLIENTE
```

Clique:

New File

Crie exatamente:

package.json

Ele fica na raiz, igual ao `index.html`.

Então teremos:

```
CASE-PAINEL-CLIENTE
|
├─ data/
├─ src/
├─ index.html
├─ styles.css
├─ package.json      ← AQUI
├─ 01-INCEPTION.md
├─ 02-PLAN.md
└─ AGENTS.md
```

PASSO 3 — Coloque isto no `package.json`

Copie exatamente:

```
{
  "name": "case-painel-cliente",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "test": "node --test"
  }
}
```

Salve:

Command + S

PASSO 4 — Entenda para que serve

O `package.json` é como uma ficha de configuração do projeto JavaScript.

Esta parte:

```
"name": "case-painel-cliente"
```

é o nome.

Esta:

```
"private": true
```

significa que não queremos publicar isso como pacote no npm.

Esta:

```
"type": "module"
```

é muito importante porque nosso `domain.js` utiliza:

```
export
```

e nossos testes utilizarão:

```
import
```

Então estamos dizendo ao Node:

este projeto trabalha com módulos JavaScript.

E:

```
"scripts": {  
  "test": "node --test"  
}
```

cria um comando fácil:

```
npm test
```

que significa:

```
npm test  
  ↓  
node --test
```

↓
procure e execute os testes

PASSO 5 — Criar a pasta `tests`

Agora no lado esquerdo do VS Code, passe o mouse sobre:

CASE-PAINEL-CLIENTE

Clique:

New Folder

Digite:

tests

Enter.

Agora:

```
CASE-PAINEL-CLIENTE
|
├─ data/
├─ src/
├─ tests/      ← NOVA
├─ index.html
├─ styles.css
├─ package.json
├─ 01-INCEPTION.md
├─ 02-PLAN.md
└─ AGENTS.md
```

PASSO 6 — Criar o arquivo de testes

Clique com o botão direito em:

tests

Depois:

New File

Digite exatamente:

```
domain.test.js
```

Agora:

```
tests/  
└─ domain.test.js
```

Abra esse arquivo.

PASSO 7 — Coloque este código

Copie **tudo**:

```
import test from "node:test";  
import assert from "node:assert/strict";  
  
import {  
  calcularPatrimonioTotal,  
  calcularPatrimonioPorInstituicao,  
  calcularAlocacao,  
  identificarOportunidadeCrossSell  
} from "../src/domain.js";  
  
const instituicoes = [  
  {  
    nome: "Itau",  
    posicoes: [  
      {  
        produto: "CDB",  
        categoria: "Renda Fixa",  
        valor: 420000  
      },  
      {  
        produto: "Acoes",  
        categoria: "Renda Variavel",  
        valor: 180000  
      },  
      {  
        produto: "Fundo de Credito",  
        categoria: "Fundos",  
        valor: 200000  
      }  
    ]  
  },  
]
```

```

{
  nome: "BTG Pactual",
  posicoes: [
    {
      produto: "Tesouro Direto",
      categoria: "Renda Fixa",
      valor: 350000
    },
    {
      produto: "Fundo Multimercado",
      categoria: "Fundos",
      valor: 400000
    },
    {
      produto: "Dolar",
      categoria: "Cambio",
      valor: 200000
    }
  ]
},

{
  nome: "XP Investimentos",
  posicoes: [
    {
      produto: "LCI",
      categoria: "Renda Fixa",
      valor: 300000
    },
    {
      produto: "Acoes Brasileiras",
      categoria: "Renda Variavel",
      valor: 200000
    },
    {
      produto: "Fundo Imobiliario",
      categoria: "Fundos",
      valor: 100000
    }
  ]
}
];

test("calcula o patrimonio total corretamente", () => {

  const resultado =
    calcularPatrimonioTotal(instituicoes);

```

```

    assert.equal(
      resultado,
      2350000
    );

  });

  test("calcula o patrimonio de cada instituicao", () => {

    const resultado =
      calcularPatrimonioPorInstituicao(
        instituicoes
      );

    assert.deepEqual(
      resultado,
      [
        {
          nome: "Itau",
          valor: 800000
        },
        {
          nome: "BTG Pactual",
          valor: 950000
        },
        {
          nome: "XP Investimentos",
          valor: 600000
        }
      ]
    );

  });

  test("calcula os percentuais de alocao", () => {

    const resultado =
      calcularAlocacao(instituicoes);

    const rendaFixa =
      resultado.find(
        (item) =>
          item.categoria === "Renda Fixa"
      );

    const rendaVariavel =
      resultado.find(
        (item) =>
          item.categoria === "Renda Variavel"
      );
  });

```

```

    );

    const fundos =
      resultado.find(
        (item) =>
          item.categoria === "Fundos"
      );

    const cambio =
      resultado.find(
        (item) =>
          item.categoria === "Cambio"
      );

    assert.equal(
      rendaFixa.percentual,
      45.53
    );

    assert.equal(
      rendaVariavel.percentual,
      16.17
    );

    assert.equal(
      fundos.percentual,
      29.79
    );

    assert.equal(
      cambio.percentual,
      8.51
    );
  });

  test("identifica oportunidade de diversificacao internacional", () => {

    const alocacao =
      calcularAlocacao(instituicoes);

    const oportunidade =
      identificarOportunidadeCrossSell(
        alocacao
      );

    assert.equal(
      oportunidade.titulo,
      "Diversificacao internacional"
    );
  });

```

```

    );

    assert.equal(
      oportunidade.produtoSugerido,
      "Produtos com exposicao internacional"
    );

  });

  test("rejeita valor patrimonial negativo", () => {

    const dadosInvalidos = [
      {
        nome: "Instituicao Teste",
        posicoes: [
          {
            produto: "Produto Teste",
            categoria: "Renda Fixa",
            valor: -100
          }
        ]
      }
    ];

    assert.throws(
      () =>
        calcularPatrimonioTotal(
          dadosInvalidos
        ),
      /Posicao patrimonial invalida/
    );

  });

  test("rejeita lista vazia de instituicoes", () => {

    assert.throws(
      () =>
        calcularPatrimonioTotal([]),
      /lista de instituicoes/
    );

  });

```

Salve:

Command + S

PASSO 8 — O que são esses testes?

Imagine que nós perguntamos ao computador:

"Quanto deveria dar a soma?"

Nós sabemos que:

```
800.000
+
950.000
+
600.000
-----
2.350.000
```

Então o teste:

```
assert.equal(
  resultado,
  2350000
);
```

significa:

eu espero que o resultado seja exatamente 2.350.000.

Se vier:

2350000

o teste passa:

✓ PASSOU

Se por algum erro nosso JavaScript retornar:

2250000

o teste falha:

✗ FALHOU

PASSO 9 — Teste das instituições

Também estamos verificando:

```
Itau
↓
R$ 800.000

BTG
↓
R$ 950.000

XP
↓
R$ 600.000
```

Então não estamos apenas acreditando que nosso código funciona.

Estamos **provando automaticamente**.

PASSO 10 — Teste da alocação

Estamos verificando:

```
Renda Fixa
45,53%

Renda Variável
16,17%

Fundos
29,79%

Câmbio
8,51%
```

Se alguém mudar o código futuramente e quebrar uma conta, o teste pode descobrir.

PASSO 11 — Teste do cross-sell

Este teste:

```
assert.equal(
  oportunidade.titulo,
  "Diversificacao internacional"
);
```

verifica nossa regra:

```
Câmbio = 8,51%
```

```
8,51 < 10
```

```
↓
```

```
Diversificação internacional
```

PASSO 12 — Também testamos erros

Esse trecho:

```
valor: -100
```

é propositalmente errado.

Queremos confirmar que nosso programa rejeita:

```
R$ -100
```

Então:

```
assert.throws(...)
```

significa:

espero que essa situação gere um erro.

Isso é muito bom para o case porque o critério técnico cita justamente tratamento de entrada inválida.

PASSO 13 — Agora executar os testes

Abra o terminal.

Certifique-se de estar:

```
case-painel-cliente %
```

Digite:

```
npm test
```

Enter.

PASSO 14 — O que queremos ver

Deverá aparecer algo semelhante a:

```
> case-painel-cliente@1.0.0 test
> node --test
```

Depois vários testes:

- ✓ calcula o patrimonio total corretamente
- ✓ calcula o patrimonio de cada instituicao
- ✓ calcula os percentuais de alocao
- ✓ identifica oportunidade de diversificacao internacional
- ✓ rejeita valor patrimonial negativo
- ✓ rejeita lista vazia de instituicoes

E no final algo parecido com:

```
tests 6
pass 6
fail 0
```

O mais importante é:

```
pass 6
fail 0
```

Ou:

```
✓ 6 testes passaram
✗ 0 testes falharam
```

PASSO 15 — SE DER ERRO

Se aparecer:

```
FAIL
```

ou:

```
ERR_MODULE_NOT_FOUND
```

ou qualquer texto vermelho:

não faça commit ainda.

Me mande um print do terminal.

Eu vou corrigir com você.

Se aparecer:

```
pass 6  
fail 0
```

pode continuar.

PASSO 16 — Agora vamos registrar no Git

Digite:

```
git status
```

Devemos ter aproximadamente:

```
Untracked files:  
  
package.json  
tests/
```

Perfeito.

PASSO 17 — Adicionar os arquivos

Digite:

```
git add package.json tests/domain.test.js
```

Enter.

Depois:

```
git status
```

Queremos:

Changes to be committed:

```
new file: package.json
new file: tests/domain.test.js
```

PASSO 18 — Criar o oitavo commit

Digite:

```
git commit -m "test: adiciona testes das regras de negocio"
```

Enter.

Agora nosso histórico fica:

```
COMMIT 1
Concepção

↓

COMMIT 2
Planejamento

↓

COMMIT 3
Dados fictícios

↓

COMMIT 4
Regras de negócio

↓

COMMIT 5
Estrutura HTML

↓

COMMIT 6
```

Estilo visual

↓

COMMIT 7
Integração

↓

COMMIT 8
Testes

Isso continua seguindo a exigência de commits incrementais e evolução real.

PASSO 19 — Subir para o GitHub

Digite:

```
git push
```

Enter.

Espere terminar.

PASSO 20 — Conferir novamente

Digite:

```
git status
```

Queremos:

```
On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

PASSO 21 — Conferir no GitHub

Atualize o GitHub:

```
Command + R
```

Agora esperamos:

```
case-painel-cliente/
├── data/
├── src/
├── tests/ ✓
├── index.html
├── styles.css
├── package.json ✓
├── 01-INCEPTION.md
├── 02-PLAN.md
└── AGENTS.md
```

No topo:

8 Commits

Entre em:

tests/

e confira:

domain.test.js

PARE NESSA ETAPA

Quando terminarmos esta parte teremos:

- ✓ Painei funcional
- ✓ JSON
- ✓ HTML
- ✓ CSS
- ✓ JavaScript
- ✓ cálculos
- ✓ cross-sell
- ✓ tratamento de erros
- ✓ testes automáticos
- ✓ 8 commits

Mas ainda temos partes **obrigatórias do case** que não podemos esquecer.

Depois desta etapa vamos fazer, nesta ordem:

ETAPA 10

↓

Atualizar 02-PLAN.md
registrando uma decisão/rejeição real da IA

ETAPA 11

↓

Atualizar AGENTS.md
skill completa e específica do projeto

ETAPA 12

↓

Criar 03-OPERATIONS.md
como executar, testar e o que faltaria para produção

ETAPA 13

↓

Criar README.md
mapa de todos os artefatos

ETAPA 14

↓

Criar /historico
e colocar nossa conversa com IA

ETAPA 15

↓

Revisão completa contra o enunciado

ETAPA 16

↓

Adicionar rodfercar como colaborador somente leitura

ETAPA 17

↓

Revisão final do GitHub e entrega

Essas etapas são importantes porque o case **não avalia apenas o site**. Ele exige explicitamente

`03-OPERATIONS.md`, uma skill específica e evolutiva, histórico completo das conversas e

`README.md`.

Faça agora **até** `npm test` **primeiro**. Se os seis testes passarem, continue até o oitavo commit e `git push`. Se qualquer teste falhar, pare exatamente ali e me mande o print.





Sim. Pelos seus prints, **a etapa de testes deu certo:**

```
tests 6
pass 6
fail 0
```

Então agora temos:

- ✓ Painel funcionando
- ✓ Dados fictícios
- ✓ Cálculos
- ✓ Cross-sell
- ✓ Tratamento de erros
- ✓ Testes automáticos
- ✓ Tudo no GitHub

Agora **não vamos criar código novo**. A próxima etapa é obrigatória no case: **evoluir o AGENTS.md**, porque eles pedem que a skill acompanhe o desenvolvimento e reflita o projeto real.

ETAPA 10 — ATUALIZAR A SKILL AGENTS.md

Nesta etapa **não crie um arquivo novo**.

Vamos atualizar o arquivo que você já possui:

```
AGENTS.md
```

No final, faremos um commit **somente dessa atualização**, para mostrar claramente que a skill evoluiu.

PASSO 1 — Abra o AGENTS.md

No VS Code, na lateral esquerda, clique:

```
AGENTS.md
```

Hoje ele está na raiz:

```
CASE-PAINEL-CLIENTE
|
├─ data/
├─ src/
├─ tests/
|
```

```
| 01-INCEPTION.md
| 02-PLAN.md
| AGENTS.md      ← ABRA ESTE
| index.html
| package.json
| styles.css
```

PASSO 2 — Apague o conteúdo antigo

Clique dentro do `AGENTS.md`.

Use:

Command + A

para selecionar tudo.

Depois:

Delete

O arquivo ficará vazio.

Isso é proposital: vamos substituir a versão inicial por uma versão que representa **o projeto que realmente construímos**.

PASSO 3 — Copie e cole este conteúdo completo

Cole isto dentro do `AGENTS.md`:

```
# AGENTS.md

## Contexto e propósito do projeto

Este repositório contém um protótipo de um Painel do Cliente para um Advisor de uma gestora de investimentos.

O objetivo da aplicação é reunir em uma única tela informações fictícias de um cliente, incluindo:

- dados pessoais e de contato;
- patrimônio distribuído entre diferentes instituições;
- patrimônio total consolidado;
- distribuição da carteira por categoria;
- histórico de relacionamento;
- oportunidade de cross-sell.
```

O projeto foi desenvolvido como parte de um case técnico e deve permanecer simples, legível e fácil de explicar em uma entrevista.

Stack utilizada

O projeto utiliza:

- HTML;
- CSS;
- JavaScript puro;
- JSON para os dados fictícios;
- Node.js apenas para execução dos testes automatizados;
- `node:test` para os testes.

Não utilizar frameworks de frontend neste projeto.

Não adicionar dependências externas sem uma necessidade clara.

Estrutura atual do projeto

```
/ |—— data/ | |—— client.json | |—— src/ | |—— app.js | |—— domain.js | |——
tests/ | |—— domain.test.js | |—— index.html |—— styles.css |—— package.json |——
01-INCEPTION.md |—— 02-PLAN.md |—— 03-OPERATIONS.md |—— AGENTS.md |——
README.md |—— historico/
```

Alguns arquivos da estrutura acima ainda podem ser criados nas etapas finais do projeto.

Responsabilidade de cada arquivo

`data/client.json`

Contém todos os dados fictícios utilizados pelo painel.

A estrutura principal possui:

- `cliente`;
- `instituicoes`;
- `interacoes`.

Cada instituição possui:

- `nome`;
- `posicoes`.

Cada posição possui:

- `produto`;
- `categoria`;
- `valor`.

Não adicionar patrimônio total pronto no JSON.

O patrimônio deve continuar sendo calculado pelas funções de domínio.

`src/domain.js`

Contém as regras de negócio da aplicação.

Atualmente é responsável por:

- validar instituições e posições;
- calcular patrimônio total;
- calcular patrimônio por instituição;
- calcular alocação por categoria;
- calcular percentuais;
- identificar a oportunidade de cross-sell.

Este arquivo NÃO deve:

- acessar elementos HTML;
- utilizar `document`;
- executar `fetch`;
- criar componentes visuais.

As funções de domínio devem continuar independentes da interface.

`src/app.js`

É responsável pela integração entre:

- dados JSON;
- regras de negócio;
- página HTML.

Este arquivo pode:

- buscar `client.json`;
- chamar funções de `domain.js`;
- formatar moeda e datas;
- criar elementos HTML;
- apresentar os resultados;
- apresentar mensagens de erro.

Evitar colocar cálculos patrimoniais diretamente neste arquivo.

Os cálculos devem permanecer em `domain.js`.

`index.html`

Contém a estrutura semântica da página.

Os valores patrimoniais e dados do cliente não devem ser escritos de forma fixa no HTML.

O JavaScript deve preencher os elementos utilizando seus respectivos IDs.

`styles.css`

Contém somente apresentação visual.

Atualmente define:

- layout em cards;
- grid da página;
- responsividade;
- barras de alocação;
- estilo do histórico;
- destaque da oportunidade de cross-sell;
- estilo da mensagem de erro.

Não colocar regras de negócio no CSS.

`tests/domain.test.js`

Contém testes automatizados das regras de negócio.

Os testes atuais verificam:

- patrimônio total;
- patrimônio por instituição;
- percentuais de alocação;
- oportunidade de cross-sell;
- rejeição de valor negativo;
- rejeição de lista vazia de instituições.

Mudanças em regras de negócio devem ser acompanhadas por atualização ou criação de testes.

Regra atual de cross-sell

A oportunidade atual é baseada na participação da categoria `Cambio`.

Quando a categoria `Cambio` não existe ou representa menos de 10% do patrimônio, o sistema sinaliza:

`Diversificacao internacional`

Produto sugerido:

`Produtos com exposicao internacional`

A justificativa informa que a carteira possui menos de 10% nessa categoria.

Esta regra deve permanecer determinística e fácil de explicar.

Não alterar o limite de 10% ou a lógica da recomendação sem:

1. justificar a mudança;
2. registrar a decisão na documentação;
3. atualizar os testes relacionados.

Convenções de nomenclatura

JavaScript

Utilizar `camelCase`.

Exemplos:

calcularPatrimonioTotal renderizarCliente formatarMoeda

Funções devem possuir nomes que indiquem claramente sua responsabilidade.

Evitar abreviações que dificultem a leitura.

CSS

Utilizar nomes de classes descritivos em formato semelhante a:

instituicao-card
alocacao-item
barra-preenchimento
interacao-item

Evitar classes genéricas como:

box1
coisa
item2

Tratamento de erros

As regras de domínio devem rejeitar dados patrimoniais inválidos.

Exemplos:

- lista de instituições vazia;
- posição sem categoria;
- valor que não seja numérico;
- valor patrimonial negativo;
- patrimônio total igual a zero quando necessário para cálculo percentual.

O `app.js` deve capturar erros ocorridos durante o carregamento e apresentar uma mensagem compreensível utilizando a seção:

mensagem-erro

Não utilizar blocos `catch` vazios.

Execução da aplicação

Na raiz do projeto, executar:

```
python3 -m http.server 5500
```

Depois acessar no navegador:

<http://localhost:5500>

Não abrir o `index.html` diretamente pelo Finder, pois a aplicação utiliza `fetch` para carregar o JSON.

Execução dos testes

Na raiz do projeto:

```
npm test
```

O resultado esperado atualmente é:

tests 6 pass 6 fail 0

Todos os testes devem continuar passando antes de uma entrega.

Padrões que devem ser mantidos

- Separar regras de negócio da interface.
- Calcular patrimônio a partir das posições.
- Calcular percentuais a partir do patrimônio real.
- Manter dados fictícios separados do código.
- Manter funções pequenas e com responsabilidades claras.
- Manter a oportunidade de cross-sell explicável.
- Validar entradas antes de realizar cálculos.
- Criar ou atualizar testes quando uma regra de negócio mudar.
- Priorizar uma solução simples que possa ser explicada pelo candidato.

Antipadrões a evitar

Não:

- adicionar React, Vue, Angular ou outro framework sem necessidade;
- adicionar biblioteca de gráficos apenas para as barras atuais;
- adicionar banco de dados;
- adicionar autenticação;
- criar backend sem necessidade;
- hardcodar `2350000` no HTML como patrimônio total;
- hardcodar os percentuais das categorias;
- duplicar regras de negócio em `app.js`;
- realizar cálculos patrimoniais dentro do HTML;
- remover validações para simplificar o código;
- adicionar dependências apenas para deixar o projeto mais sofisticado.

Decisões arquiteturais que não devem ser contrariadas

A aplicação continuará sendo um protótipo de uma única página.

Os dados continuarão mockados em JSON.

A arquitetura deve permanecer:

client.json ↓ app.js ↓ domain.js ↓ index.html

Mais precisamente:

client.json ↓ dados

domain.js ↓ regras e cálculos

app.js ↓ integração e renderização

index.html + styles.css ↓ interface

Uma mudança nessa estrutura deve possuir uma justificativa técnica documentada.

Evolução desta skill

Esta skill foi criada antes da implementação com regras iniciais e foi refinada

depois que a estrutura real da aplicação, as funções de domínio, a integração e os testes foram definidos.

Ela deve continuar sendo atualizada caso novas decisões importantes sejam tomadas.

Instrução considerada e não incluída

Foi considerada a possibilidade de exigir que todas as funções JavaScript utilizassem arrow functions.

Decidi não incluir essa regra.

O código atual utiliza declarações de função com `function`, que são legíveis e adequadas para o tamanho do projeto.

Obrigar a mudança apenas por preferência de estilo não traria benefício técnico e aumentaria alterações desnecessárias no código já funcional.

PASSO 4 — Salve

Pressione:

Command + S

Agora o `AGENTS.md` está muito mais completo.

PASSO 5 — Por que fizemos isso?

No começo do projeto nossa skill dizia basicamente:

Use HTML
Use CSS
Use JavaScript
Não complique

Naquele momento isso fazia sentido porque ainda não tínhamos código.

Agora sabemos que existe:

src/domain.js
src/app.js
tests/domain.test.js
data/client.json

E sabemos exatamente a responsabilidade de cada um.

Então a skill **evoluiu com o projeto**:

```
SKILL INICIAL
  ↓
projeto foi construído
  ↓
aprendemos a arquitetura real
  ↓
SKILL REFINADA
```

Isso é exatamente algo que o case pede: a skill deve ser específica do projeto e evoluir conforme o desenvolvimento.

PASSO 6 — Repare em uma exigência muito importante

No final colocamos:

```
## Instrução considerada e não incluída
```

Por quê?

Porque o próprio case pede que, ao final, exista **uma instrução que você considerou incluir e decidiu não incluir**, com justificativa.

Nós usamos:

```
Obrigar arrow functions
```

e justificamos que seria apenas preferência de estilo e não benefício técnico.

Isso cumpre exatamente essa parte.

PASSO 7 — Conferir o Git

Agora abra o terminal.

Digite:

```
git status
```

Esperamos algo parecido com:

```
Changes not staged for commit:
```

```
    modified: AGENTS.md
```

Perfeito.

Nesta etapa queremos somente `AGENTS.md` .

PASSO 8 — Conferir o que mudou

Antes do commit, execute:

```
git diff AGENTS.md
```

Vai aparecer bastante coisa.

Isso é normal.

As linhas antigas/removidas aparecem com:

```
-
```

As novas:

```
+
```

Você não precisa alterar nada.

É apenas uma forma de conferir:

estou realmente modificando apenas o AGENTS.md.

PASSO 9 — Adicionar ao commit

Digite:

```
git add AGENTS.md
```

Enter.

Depois:

```
git status
```

Queremos:

Changes to be committed:

modified: AGENTS.md

PASSO 10 — Criar o nono commit

Digite:

```
git commit -m "docs: refina skill com estrutura atual do projeto"
```

Enter.

Agora seu histórico terá:

```
COMMIT 1  
Concepção
```

↓

```
COMMIT 2  
Planejamento
```

↓

```
COMMIT 3  
Dados
```

↓

```
COMMIT 4  
Regras
```

↓

```
COMMIT 5  
HTML
```

↓

```
COMMIT 6  
CSS
```

↓

```
COMMIT 7
Integração

↓

COMMIT 8
Testes

↓

COMMIT 9
Refinamento da skill
```

Esse commit separado é particularmente útil porque o avaliador consegue enxergar claramente:

```
AGENTS inicial
↓
...
↓
AGENTS refinado
```

PASSO 11 — Subir para o GitHub

Digite:

```
git push
```

Enter.

Espere terminar.

PASSO 12 — Conferir

Agora:

```
git status
```

Queremos:

```
On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Perfeito.

PASSO 13 — Conferir no GitHub

Volte para o GitHub.

Atualize:

Command + R

Agora deverá aparecer:

9 Commits

Clique:

AGENTS.md

E confira se aparecem seções como:

Contexto e propósito do projeto

Stack utilizada

Estrutura atual do projeto

Responsabilidade de cada arquivo

Regra atual de cross-sell

Tratamento de erros

Execução

Testes

Padrões

Antipadrões

Decisões arquiteturais

Evolução desta skill

Instrução considerada e não incluída

Se tudo isso aparecer:

✓ ETAPA 10 FINALIZADA

O QUE VEM DEPOIS?

Depois dessa etapa nós vamos fazer algo igualmente importante.

ETAPA 11 — Atualizar o 02-PLAN.md

Porque o case não quer apenas:

“A IA me deu código e eu aceitei.”

Ele quer saber:

O que a IA sugeriu?
↓
Você concordou?
↓
Você rejeitou alguma coisa?
↓
Por quê?

O documento pede explicitamente que, durante o desenvolvimento, seja indicado onde propostas da IA foram corrigidas ou descartadas.

Nós temos uma decisão **real** para registrar: no início houve uma sugestão de usar Node também como servidor local; durante a implementação mantivemos o projeto mais simples e usamos `python3 -m http.server` apenas para servir os arquivos e Node somente para os testes.

Depois faremos:

ETAPA 11
02-PLAN.md – registrar decisões reais

↓

ETAPA 12
03-OPERATIONS.md

↓

ETAPA 13
README.md

↓

ETAPA 14
/historico

↓

ETAPA 15
revisão completa

↓

ETAPA 16
adicionar rodfercar

↓

ETAPA 17
entrega final

Faça agora **somente a atualização do** `AGENTS.md`, **o commit e o push**. Quando o GitHub mostrar **9 commits**, seguimos para a atualização do `02-PLAN.md`, novamente passo por passo.



Perfeito. Como você já atualizou o `AGENTS.md` e subiu para o GitHub, agora vamos para a **próxima etapa obrigatória do case: atualizar o** `02-PLAN.md` **com uma decisão real tomada durante o desenvolvimento**.

O case pede explicitamente que você registre onde uma proposta da IA foi **corrigida ou descartada e por quê**.

ETAPA 11 — ATUALIZAR O `02-PLAN.md`

Aqui **não vamos criar arquivo novo**.

Vamos modificar:

`02-PLAN.md`

O objetivo é mostrar ao avaliador:

IA sugeriu uma possibilidade
↓
eu analisei
↓
decidi não seguir

↓
expliquei o motivo

Isso é muito importante porque eles avaliam justamente seu **crivo crítico sobre a IA**, não apenas se você copiou código.

PASSO 1 — Abrir o arquivo

No VS Code, na lateral esquerda, clique em:

02-PLAN.md

Sua estrutura deve estar parecida com:

```
CASE-PAINEL-CLIENTE
|
├─ data/
├─ src/
├─ tests/
|
├─ 01-INCEPTION.md
├─ 02-PLAN.md      ← ABRA ESTE
├─ AGENTS.md
├─ index.html
├─ package.json
└─ styles.css
```

PASSO 2 — Ir até o final

Role o arquivo até a última parte.

Você provavelmente encontrará esta seção que criamos anteriormente:

```
## Registro de propostas da IA corrigidas ou rejeitadas
```

Esta seção será atualizada durante o desenvolvimento.

Até o momento, ainda não houve uma proposta técnica de implementação rejeitada após o início da construção.

Sempre que uma sugestão da IA for considerada inadequada, registrarei:

- o que a IA sugeriu;
- qual problema identifiquei;

- qual decisão tomei;
- por que escolhi a alternativa.

Agora esse texto está desatualizado.

Porque **já temos uma decisão real que podemos registrar.**

PASSO 3 — O que realmente aconteceu?

No começo do projeto, durante a orientação, foi considerada a possibilidade de utilizar **Node.js também para servir a aplicação localmente.**

Mas durante a construção percebemos que isso era desnecessário.

Nosso projeto é:

```
HTML
CSS
JavaScript
JSON
```

Não temos backend.

Então escolhemos:

```
python3 -m http.server 5500
```

para simplesmente servir os arquivos no navegador.

E deixamos o Node.js para:

```
npm test
```

Ou seja:

PROPOSTA CONSIDERADA

Node.js para servidor local

↓

ANALISEI

↓

não precisamos de backend

↓

DECISÃO

Python http.server para execução local

Node.js somente para testes

Essa é uma decisão pequena, mas **real e defensável**.

PASSO 4 — Substituir a seção antiga

No `02-PLAN.md`, encontre:

```
## Registro de propostas da IA corrigidas ou rejeitadas
```

Apague **essa seção até o final dela**.

Depois cole exatamente isto:

```
## Registro de propostas da IA corrigidas ou rejeitadas
```

```
### Decisão 1 - Servidor local da aplicação
```

Durante a orientação inicial com IA, foi considerada a possibilidade de utilizar Node.js também para servir os arquivos da aplicação localmente.

Ao avançar na implementação, analisei que o protótipo não possui backend, banco de dados ou rotas de servidor. A aplicação precisa apenas servir arquivos HTML, CSS, JavaScript e JSON para que o navegador consiga utilizar `fetch``.

Por esse motivo, decidi não adicionar um servidor Node.js ao projeto.

Para executar o protótipo localmente, mantive uma solução mais simples:

```
python3 -m http.server 5500
```

O Node.js permaneceu no projeto apenas para executar os testes automatizados através do comando:

```
npm test
```

```
### Motivo da decisão
```

Adicionar um servidor Node.js exclusivo para servir arquivos estáticos aumentaria a quantidade de código e arquivos sem resolver um problema necessário para este protótipo.

Preferi manter a aplicação simples, com menos dependências e mais fácil de explicar durante uma entrevista técnica.

A decisão final ficou:

- Python `http.server` para servir a aplicação localmente;
- Node.js para executar os testes;
- nenhuma camada de backend adicionada.

Essa decisão também está alinhada com o objetivo de evitar complexidade desnecessária no protótipo.

PASSO 5 — Salvar

Pressione:

Command + S

Pronto.

Agora o `02-PLAN.md` mostra uma decisão real do desenvolvimento.

PASSO 6 — Por que isso é importante?

Imagine que o recrutador abra seu GitHub.

Ele vê:

01-INCEPTION.md

↓

o que Bernardo entendeu

02-PLAN.md

↓

como Bernardo planejou

commits

↓

como o projeto evoluiu

Registro da IA

↓

Bernardo não aceitou tudo automaticamente

Isso demonstra exatamente:

```
"IA sugeriu"  
  ↓  
"eu pensei"  
  ↓  
"eu tomei uma decisão"
```

É isso que eles querem observar.

PASSO 7 — NÃO invente outras rejeições

Isso é importante.

Não precisamos escrever:

```
"rejeitei React"  
"rejeitei banco"  
"rejeitei API"  
"rejeitei framework X"
```

se isso não aconteceu de verdade.

O próprio case quer um histórico verdadeiro, inclusive tentativas e mudanças de direção.

Uma decisão verdadeira é melhor do que cinco inventadas.

PASSO 8 — Conferir o Git

Abra o terminal.

Digite:

```
git status
```

Você deverá encontrar algo parecido com:

```
Changes not staged for commit:  
  
  modified: 02-PLAN.md
```

Perfeito.

Queremos **somente esse arquivo modificado nesta etapa**.

PASSO 9 — Ver exatamente o que mudou

Digite:

```
git diff 02-PLAN.md
```

Enter.

Você verá linhas com:

```
-
```

e:

```
+
```

Isso não significa erro.

O Git está mostrando:

```
- conteúdo antigo  
+ conteúdo novo
```

Confira apenas se a alteração está relacionada à seção:

```
Registro de propostas da IA corrigidas ou rejeitadas
```

PASSO 10 — Adicionar ao Git

Digite:

```
git add 02-PLAN.md
```

Enter.

Depois:

```
git status
```

Queremos:

```
Changes to be committed:
```

modified: 02-PLAN.md

Perfeito.

PASSO 11 — Criar o próximo commit

Digite:

```
git commit -m "docs: registra decisao tomada durante desenvolvimento"
```

Enter.

Agora seu histórico estará aproximadamente:

COMMIT 1
Concepção

↓

COMMIT 2
Planejamento

↓

COMMIT 3
Dados

↓

COMMIT 4
Regras

↓

COMMIT 5
HTML

↓

COMMIT 6
CSS

↓

COMMIT 7
Integração

↓

COMMIT 8

Testes

↓

COMMIT 9

Refinamento da skill

↓

COMMIT 10

Decisão registrada no plano

Esse é um histórico bem coerente para o case.

PASSO 12 — Subir para o GitHub

Digite:

```
git push
```

Enter.

Aguarde terminar.

PASSO 13 — Conferir se está tudo limpo

Digite:

```
git status
```

Queremos:

```
On branch main
```

```
Your branch is up to date with 'origin/main'.
```

```
nothing to commit, working tree clean
```

Principalmente:

```
nothing to commit, working tree clean
```

Isso significa:

- ✓ alteração salva
- ✓ commit criado
- ✓ GitHub atualizado

PASSO 14 — Conferir no GitHub

Volte para seu repositório no GitHub.

Atualize:

Command + R

Provavelmente agora aparecerá:

10 Commits

Clique em:

02-PLAN.md

Role até o final.

Confira se aparece:

Registro de propostas da IA corrigidas ou rejeitadas

Decisão 1 - Servidor local da aplicação

Se aparecer, terminou.

ETAPA 11 CONCLUÍDA

Nesse ponto teremos:

- ✓ 01-INCEPTION.md
- ✓ 02-PLAN.md
 - └─ planejamento
 - └─ decisão/rejeição da IA
- ✓ AGENTS.md
 - └─ skill atualizada

- ✓ aplicação funcional
- ✓ testes
- ✓ histórico incremental de commits

PRÓXIMA ETAPA — 03-OPERATIONS.md

Essa será nossa **ETAPA 12**.

Ela é obrigatória. O case pede que esse arquivo explique:

1. como executar a aplicação;
2. como testar;
3. o que ainda faltaria para levar o projeto para produção.

Vamos criar:

03-OPERATIONS.md

e nele teremos algo organizado assim:

```
Como executar
↓
python3 -m http.server 5500

Como acessar
↓
http://localhost:5500

Como testar
↓
npm test

Resultado esperado
↓
6 testes passando

O que faltaria para produção
↓
API real
autenticação
segurança
logs
monitoramento
deploy
```

dados reais
etc.

Faça primeiro somente esta atualização do 02-PLAN.md , commit e push. Depois disso, a próxima etapa é criar o 03-OPERATIONS.md do zero, e eu te passo exatamente **onde criar, o que copiar e colar, como conferir, fazer o commit e subir.**



Perfeito. Como você já atualizou o 02-PLAN.md , fez o commit e subiu para o GitHub, agora vamos para a **ETAPA 12 — criar o 03-OPERATIONS.md** .

Esse arquivo é obrigatório no case. Ele deve explicar **como executar a aplicação, como testar e o que ainda faltaria para levar o protótipo para produção.**

Vamos fazer exatamente como antes: criar arquivo → copiar conteúdo → conferir → testar → commit → push.

ETAPA 12 — CRIAR 03-OPERATIONS.md

No final, sua estrutura ficará assim:

```
CASE-PAINEL-CLIENTE
|
├─ data/
|   └─ client.json
|
├─ src/
|   ├── app.js
|   └─ domain.js
|
├─ tests/
|   └─ domain.test.js
|
├─ 01-INCEPTION.md
├─ 02-PLAN.md
├─ 03-OPERATIONS.md    ← NOVO
├─ AGENTS.md
├─ index.html
├─ package.json
└─ styles.css
```

PASSO 1 — Criar o arquivo

Abra o VS Code.

Na lateral esquerda, passe o mouse sobre:

CASE-PAINEL-CLIENTE

Clique no ícone:

New File

Crie exatamente:

03-OPERATIONS.md

Atenção ao nome:

03-OPERATIONS.md

Não:

operations.txt

nem:

03_OPERATIONS.md

Queremos exatamente:

03-OPERATIONS.md

PASSO 2 — Abra o arquivo

Clique:

03-OPERATIONS.md

Ele estará vazio.

Agora vamos colocar o conteúdo.

PASSO 3 — Copie e cole tudo abaixo

03 - Operação

Objetivo deste documento

Este documento descreve como executar e testar o protótipo do Painel do Cliente em ambiente local e apresenta os principais pontos que ainda precisariam ser tratados antes de uma utilização em produção.

Pré-requisitos

Para executar completamente o projeto, é necessário possuir:

- um navegador moderno, como Chrome, Safari, Edge ou Firefox;
- Python 3 para servir os arquivos localmente;
- Node.js e npm para executar os testes automatizados.

O projeto não utiliza banco de dados e não possui dependências externas de frontend.

Como executar a aplicação

1. Abrir o terminal na raiz do projeto

O terminal deve estar localizado na pasta:

case-painel-cliente

É possível confirmar a localização executando:

pwd

2. Iniciar o servidor local

Execute:

python3 -m http.server 5500

O terminal deverá apresentar uma mensagem indicando que o servidor está utilizando a porta 5500.

Exemplo:

Serving HTTP on 0.0.0.0 port 5500

3. Abrir a aplicação no navegador

Acesse:

http://localhost:5500

A aplicação deverá carregar automaticamente os dados fictícios armazenados em:

data/client.json

Resultado esperado

Ao carregar corretamente, o painel deverá apresentar:

- nome do cliente;
- segmento;
- e-mail;
- telefone;
- patrimônio total consolidado;
- patrimônio de cada instituição;
- distribuição percentual da alocação;
- barras visuais de alocação;
- histórico de relacionamento;
- oportunidade de cross-sell.

Com os dados atuais, o patrimônio consolidado esperado é:

R\$ 2.350.000,00

As instituições utilizadas atualmente são:

- Itau;

```
- BTG Pactual;  
- XP Investimentos.
```

```
---
```

```
## Como encerrar a aplicação
```

No terminal em que o servidor está executando, pressionar:

Control + C

Isso encerra o servidor HTTP local.

```
---
```

```
## Como executar os testes
```

Na raiz do projeto, execute:

npm test

O projeto utiliza o módulo nativo `node:test`.

Não é necessário instalar biblioteca externa de testes.

```
---
```

```
## Resultado esperado dos testes
```

Atualmente existem 6 testes automatizados.

O resultado esperado é:

tests 6 pass 6 fail 0

Os testes verificam:

1. cálculo do patrimônio total;
2. cálculo do patrimônio por instituição;
3. cálculo dos percentuais de alocação;
4. identificação da oportunidade de diversificação internacional;
5. rejeição de valor patrimonial negativo;
6. rejeição de lista vazia de instituições.

Todos os testes devem passar antes da entrega de uma alteração.

Verificação manual recomendada

Além dos testes automatizados, é recomendado realizar uma verificação visual.

Após iniciar a aplicação, conferir:

Dados do cliente

O painel deve apresentar:

Mariana Costa Private

Patrimônio consolidado

O valor esperado é:

R\$ 2.350.000,00

Instituições

Os valores esperados são:

Itau R\$ 800.000,00

BTG Pactual R\$ 950.000,00

XP Investimentos R\$ 600.000,00

Distribuição de alocação

Os percentuais esperados atualmente são aproximadamente:

Renda Fixa: 45,53% Renda Variavel: 16,17% Fundos: 29,79% Cambio: 8,51%

Cross-sell

Como a categoria Cambio representa menos de 10% do patrimônio, o painel deve sinalizar:

Diversificacao internacional

com sugestão de produtos com exposição internacional.

Tratamento de erros

Caso o arquivo:

data/client.json

não possa ser carregado, a aplicação deverá apresentar uma mensagem de erro na interface.

As regras de domínio também rejeitam algumas entradas inválidas.

Exemplos:

- lista vazia de instituições;
- instituição sem estrutura esperada;
- posição sem categoria;
- valor que não seja numérico;
- valor patrimonial negativo;
- patrimônio igual a zero quando o percentual precisa ser calculado.

Detalhes técnicos adicionais podem ser consultados no console do navegador.

Problemas comuns

A página abre, mas os dados não aparecem

Verificar se a aplicação foi aberta através de:

`http://localhost:5500`

Não utilizar o arquivo `index.html` diretamente pelo Finder.

A aplicação utiliza `fetch` para carregar o JSON e, por isso, deve ser executada através de um servidor HTTP local.

A porta 5500 já está sendo utilizada

Caso exista outro processo utilizando a porta 5500, é possível encerrar o processo anterior ou utilizar outra porta.

Exemplo:

```
python3 -m http.server 5501
```

Nesse caso, acessar:

<http://localhost:5501>

O comando npm não existe

Verificar se Node.js e npm estão instalados:

```
node --version
```

e:

```
npm --version
```

O que faltaria para produção

Este projeto é propositalmente um protótipo e utiliza dados fictícios.

Para transformar a solução em um sistema de produção, seriam necessárias evoluções importantes.

1. Fonte de dados real

Atualmente os dados estão em:

data/client.json

Em produção seria necessário integrar o painel com APIs ou serviços responsáveis pelos dados de clientes, patrimônio e relacionamento.

2. Backend

O protótipo não possui backend.

Em produção provavelmente seria necessária uma camada de servidor para:

- acessar sistemas internos;
- consolidar informações;
- aplicar regras de autorização;
- proteger dados sensíveis;
- disponibilizar APIs para a interface.

3. Autenticação e autorização

O protótipo não possui login.

Em produção seria necessário garantir que somente usuários autorizados possam acessar informações de clientes.

Também seria necessário definir permissões de acordo com o perfil do usuário.

4. Segurança

Seria necessário avaliar e implementar controles como:

- comunicação HTTPS;
- proteção de dados pessoais e financeiros;
- controle de acesso;
- validação de entradas;
- gerenciamento seguro de credenciais;
- políticas de segurança e privacidade;
- adequação às regras aplicáveis de proteção de dados.

5. Observabilidade

Uma versão de produção deveria possuir:

- logs estruturados;
- monitoramento;
- métricas;
- alertas;
- rastreamento de erros.

Isso permitiria identificar falhas e acompanhar o funcionamento da aplicação.

6. Testes adicionais

Os testes atuais cobrem as principais regras de negócio.

Uma versão de produção deveria incluir também:

- testes de integração;
- testes da interface;
- testes de API;
- testes de segurança;
- mais cenários de dados inválidos;
- testes de acessibilidade.

7. Persistência de dados

Caso o sistema permita registrar novas interações, editar informações ou armazenar recomendações, seria necessária uma solução de persistência adequada.

O protótipo atual é somente de leitura.

8. Regra de cross-sell

A regra atual é propositalmente simples e demonstrativa.

Em produção, a oportunidade comercial deveria utilizar regras de negócio validadas pela empresa e considerar fatores adicionais, por exemplo:

- perfil de risco;
- objetivos do cliente;

- horizonte de investimento;
- produtos já contratados;
- adequação e elegibilidade;
- regras internas da instituição.

9. Deploy e CI/CD

Seria necessário definir uma estratégia de publicação da aplicação.

Também seria recomendado criar uma pipeline automatizada contendo etapas como:

commit ↓ testes ↓ validações ↓ build ↓ deploy

Limitações conhecidas do protótipo

As principais limitações atuais são:

- apenas um cliente fictício;
- dados estáticos;
- ausência de autenticação;
- ausência de backend;
- ausência de banco de dados;
- ausência de integração com instituições reais;
- regra de cross-sell simplificada;
- aplicação apenas para visualização.

Essas limitações são intencionais para manter o escopo adequado ao objetivo do case.

PASSO 4 — Salvar

Pressione:

Command + S

Agora o arquivo está criado.

PASSO 5 — Entenda o que acabamos de fazer

Esse arquivo responde três perguntas principais.

Pergunta 1

Como eu rodo seu projeto?

Resposta:

```
python3 -m http.server 5500
```

e:

```
http://localhost:5500
```

Pergunta 2

Como eu testo seu projeto?

Resposta:

```
npm test
```

Resultado:

```
tests 6  
pass 6  
fail 0
```

Pergunta 3

Isso já poderia ir para um banco ou gestora de verdade?

Resposta:

Não.

Porque ainda precisaríamos de coisas como:

```
API real  
autenticação  
segurança  
backend  
dados reais  
logs  
monitoramento  
deploy  
mais testes
```

Isso mostra que você entende a diferença entre:

```
PROTÓTIPO
```

e:

```
SISTEMA DE PRODUÇÃO
```

PASSO 6 — Vamos conferir se os comandos documentados ainda funcionam

Antes de fazer o commit, abra o terminal.

Execute:

```
npm test
```

Queremos novamente:

```
tests 6  
pass 6  
fail 0
```

Se aparecer:

```
pass 6  
fail 0
```

ótimo.

PASSO 7 — Testar a execução da aplicação

Agora execute:

```
python3 -m http.server 5500
```

Abra:

```
http://localhost:5500
```

Confira rapidamente:

- ✓ Mariana Costa
- ✓ R\$ 2.350.000,00
- ✓ 3 instituições
- ✓ barras
- ✓ histórico
- ✓ cross-sell

Depois volte ao terminal e encerre:

```
Control + C
```

Pronto.

Agora provamos que o que escrevemos no `03-OPERATIONS.md` realmente corresponde ao projeto.

PASSO 8 — Conferir o Git

Digite:

```
git status
```

Você deve encontrar algo parecido com:

```
Untracked files:
```

```
03-OPERATIONS.md
```

Perfeito.

Queremos somente esse arquivo novo nesta etapa.

PASSO 9 — Adicionar o arquivo

Digite:

```
git add 03-OPERATIONS.md
```

Enter.

Depois:

```
git status
```

Agora esperamos:

Changes to be committed:

new file: 03-OPERATIONS.md

Perfeito.

PASSO 10 — Fazer o próximo commit

Digite:

```
git commit -m "docs: adiciona instrucoes de operacao e producao"
```

Enter.

Agora seu histórico vai ficar aproximadamente:

```
COMMIT 1
Concepção

↓

COMMIT 2
Planejamento

↓

COMMIT 3
Dados

↓

COMMIT 4
Regras

↓

COMMIT 5
HTML

↓

COMMIT 6
CSS

↓
```

COMMIT 7
Integração

↓

COMMIT 8
Testes

↓

COMMIT 9
Skill refinada

↓

COMMIT 10
Decisão registrada

↓

COMMIT 11
Operação

PASSO 11 — Subir para o GitHub

Execute:

```
git push
```

Enter.

Espere terminar.

PASSO 12 — Conferir

Execute:

```
git status
```

Queremos:

```
On branch main
```

```
Your branch is up to date with 'origin/main'.
```

```
nothing to commit, working tree clean
```

Se aparecer:

```
nothing to commit, working tree clean
```

está perfeito.

PASSO 13 — Conferir no GitHub

Volte ao navegador.

Atualize:

```
Command + R
```

Agora deverá aparecer:

```
03-OPERATIONS.md
```

na raiz.

Algo assim:

```
case-painel-cliente/
|
├─ data/
├─ src/
├─ tests/
|
├─ 01-INCEPTION.md
├─ 02-PLAN.md
├─ 03-OPERATIONS.md ✓
├─ AGENTS.md
├─ index.html
├─ package.json
└─ styles.css
```

Provavelmente agora teremos cerca de:

```
11 Commits
```

Clique em:

03-OPERATIONS.md

E confira se aparecem seções como:

Pré-requisitos

Como executar a aplicação

Como executar os testes

Resultado esperado

Tratamento de erros

Problemas comuns

O que faltaria para produção

Limitações conhecidas

Se aparecer:

✓ ETAPA 12 CONCLUÍDA

A PRÓXIMA ETAPA

Depois disso vamos criar um arquivo **muito importante**:

README.md

Esse é obrigatório na entrega. O case pede que exista um `README.md` na raiz apontando onde está cada artefato.

Ele será como a **página inicial do seu projeto no GitHub**.

Quando alguém entrar no repositório, em vez de aparecer:

Add a README

vai aparecer uma apresentação bonita explicando:

Painel do Cliente

O que é o projeto

Como executar

Como testar

Estrutura

Onde está 01-INCEPTION.md

Onde está 02-PLAN.md

Onde está 03-OPERATIONS.md

Onde está AGENTS.md

Onde está /historico

Tecnologias

Requisitos atendidos

Então agora faça **somente esta ETAPA 12**: criar 03-OPERATIONS.md → testar → git add → commit → git push .

Quando ele estiver aparecendo no GitHub, seguimos para a **ETAPA 13 — criar o README.md desde o zero**, do mesmo jeito, passo por passo.



Sim. Pelo que aparece nos seus prints, **essa etapa está correta.** ✓

Você fez o que precisava no 03-OPERATIONS.md e também confirmou que a aplicação continua funcionando no navegador.

Pelo seu VS Code temos:

- ✓ 03-OPERATIONS.md criado na raiz
- ✓ conteúdo inserido
- ✓ servidor local funcionando
- ✓ painel carregando normalmente
- ✓ dados do cliente aparecendo
- ✓ patrimônio aparecendo
- ✓ alocação funcionando
- ✓ histórico funcionando
- ✓ cross-sell funcionando

Para esta etapa, **era isso mesmo**. Agora falta somente registrar o 03-OPERATIONS.md no Git e enviar para o GitHub.

PASSO 1 — Parar o servidor

No seu print, o terminal ainda está rodando:

```
python3 -m http.server 5500
```

Por isso o terminal está mostrando coisas como:

```
GET / HTTP/1.1  
GET /styles.css  
GET /src/app.js
```

Clique no terminal e pressione:

```
Control + C
```

Atenção: é **Control**, não Command.

Depois deverá voltar para algo parecido com:

```
bernardosilva@bernardo case-painel-cliente %
```

PASSO 2 — Rodar os testes mais uma vez

Antes de subir, vamos ter certeza de que não quebramos nada.

Digite:

```
npm test
```

Enter.

Queremos novamente:

```
tests 6  
pass 6  
fail 0
```

Se aparecer:

```
pass 6  
fail 0
```

pode continuar.

PASSO 3 — Ver o que o Git encontrou

Digite:

```
git status
```

Enter.

Provavelmente aparecerá:

```
Untracked files:
```

```
03-OPERATIONS.md
```

Isso quer dizer:

O Git encontrou o arquivo novo, mas ainda não colocou ele em um commit.

Está correto.

PASSO 4 — Adicionar o arquivo ao Git

Digite exatamente:

```
git add 03-OPERATIONS.md
```

Enter.

Talvez não apareça nenhuma mensagem.

Isso é normal.

PASSO 5 — Conferir

Agora digite:

```
git status
```

Queremos algo parecido com:

```
Changes to be committed:
```

```
new file: 03-OPERATIONS.md
```

Ou seja:

```
03-OPERATIONS.md
```

↓

```
pronto para entrar no commit
```

PASSO 6 — Criar o commit

Agora digite:

```
git commit -m "docs: adiciona instrucoes de operacao e producao"
```

Enter.

Deve aparecer algo parecido com:

```
[main xxxxxxx] docs: adiciona instrucoes de operacao e producao
```

```
1 file changed...
```

```
create mode 100644 03-OPERATIONS.md
```

Pronto.

O arquivo agora está oficialmente registrado no histórico.

PASSO 7 — Enviar para o GitHub

Digite:

```
git push
```

Enter.

Espere alguns segundos.

Deverá aparecer algo semelhante a:

```
To https://github.com/Mackenzie-TOTVS/case-painel-cliente.git
```

```
main -> main
```

Isso significa:

SEU MAC

|
| git push



GITHUB



PASSO 8 — Conferir se não ficou nada pendente

Digite:

```
git status
```

Queremos:

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

A frase mais importante é:

nothing to commit, working tree clean

Significa:

✓ arquivo salvo

✓ commit feito

✓ push feito

✓ nada pendente

PASSO 9 — Conferir no GitHub

Volte para o seu repositório no GitHub.

Atualize:

Command + R

Agora deverá aparecer na raiz:

```
case-painel-cliente/  
|  
├─ data/  
├─ src/  
├─ tests/  
|  
├─ 01-INCEPTION.md  
├─ 02-PLAN.md  
├─ 03-OPERATIONS.md  
├─ AGENTS.md  
├─ index.html  
├─ package.json  
└─ styles.css
```



Clique em:

03-OPERATIONS.md

e veja se o GitHub mostra o texto formatado.

Onde estamos agora

Depois desse push teremos:

- ✓ 01-INCEPTION.md
concepção
- ✓ 02-PLAN.md
planejamento + decisão com IA
- ✓ AGENTS.md
skill específica do projeto
- ✓ 03-OPERATIONS.md
execução + testes + produção
- ✓ client.json
dados
- ✓ domain.js
regras
- ✓ app.js
integração
- ✓ index.html

estrutura

✓ styles.css
aparência

✓ domain.test.js
testes

✓ aplicação funcionando

A próxima etapa será **muito importante**:

ETAPA 13 — CRIAR O `README.md`

Hoje no seu GitHub provavelmente ainda aparece:

Add a README

Vamos substituir isso por uma página inicial profissional do projeto.

O `README.md` também é **obrigatório no case**: ele deve ficar na raiz e apontar onde está cada artefato.

Ele vai mostrar algo como:

Painel do Cliente
Galapagos Capital

Sobre o projeto
Tecnologias
Requisitos atendidos
Como executar
Como testar
Estrutura do projeto

Artefatos do case:
01-INCEPTION.md
02-PLAN.md
03-OPERATIONS.md
AGENTS.md
/historico

Então agora faça apenas:

`Control + C`

`npm test`

```
git status

git add 03-OPERATIONS.md

git status

git commit -m "docs: adiciona instrucoes de operacao e producao"

git push

git status
```

Quando o `03-OPERATIONS.md` estiver aparecendo no GitHub, podemos partir direto para o **README.md**, novamente desde criar o arquivo até fazer o push.



vamos para essa etapa , me de um passo a passo quero um readme.md bem bonito



Dá para fazer de uma forma **mais profissional sem esconder o uso da IA**.

O que eu **não recomendo** é colocar apenas um resumo ou apagar trechos para parecer que você fez menos com IA. O enunciado pede explicitamente o histórico completo das interações utilizadas no desenvolvimento, em ordem cronológica, e diz para não remover tentativas ruins ou mudanças de rumo.

Mas existe uma diferença importante: **você não precisa colocar todas as conversas da sua conta do ChatGPT**. Você precisa colocar as conversas que foram utilizadas **neste case**.

E eles já sabem que a IA pode ter ajudado bastante. O case diz que o principal é observar **como você usa a IA**, e depois avaliar se você consegue explicar e alterar a própria solução.

Então vamos fazer de uma forma organizada.

ETAPA 14 — CRIAR O `/historico`

Queremos chegar nisso:

```
case-painel-cliente/
|
├─ data/
├─ src/
├─ tests/
|
```

```
├─ historico/
│   ├── 00-LEIA-ME.md
│   └── 01-chatgpt-case-painel-cliente.pdf
│
├─ 01-INCEPTION.md
├─ 02-PLAN.md
├─ 03-OPERATIONS.md
├─ AGENTS.md
├─ README.md
├─ index.html
├─ package.json
└─ styles.css
```

O PDF será a conversa original.

O `00-LEIA-ME.md` será uma pequena explicação sua para quem estiver avaliando.

PASSO 1 — Criar a pasta

No VS Code, passe o mouse sobre:

```
CASE-PAINEL-CLIENTE
```

Clique em:

```
New Folder
```

Nome:

```
historico
```

Enter.

Agora:

```
CASE-PAINEL-CLIENTE
├─
├─ data
├─ src
├─ tests
├─ historico    ← NOVA
├─
└─ ...
```

PASSO 2 — Criar um arquivo explicando o histórico

Clique com botão direito em:

historico

Depois:

New File

Nome:

00-LEIA-ME.md

PASSO 3 — Cole isto no 00-LEIA-ME.md

```
# Histórico de interações com IA
```

```
Esta pasta contém o histórico das interações com IA utilizadas durante o desenvolvimento do case técnico "Painel do Cliente".
```

```
## Ferramenta utilizada
```

```
A principal ferramenta utilizada durante o desenvolvimento foi o ChatGPT.
```

```
Utilizei a IA principalmente como apoio para:
```

- compreender os requisitos do case;
- decompor o problema em etapas;
- discutir decisões de implementação;
- gerar propostas iniciais de código;
- entender o funcionamento do código produzido;
- solucionar problemas de Git e GitHub;
- estruturar os testes;
- estruturar a documentação do projeto.

```
## Forma de trabalho
```

```
O desenvolvimento foi realizado de maneira incremental.
```

```
A cada etapa, procurei:
```

1. entender o objetivo da alteração;
2. aplicar a implementação;
3. executar a aplicação ou os testes;

4. verificar o resultado;
5. registrar a evolução através de commits;
6. seguir para a próxima etapa somente após validar a anterior.

O histórico contém momentos em que solicitei explicações mais detalhadas e mudanças na forma de orientação da IA.

Essas interações foram preservadas porque fazem parte do processo real utilizado durante o desenvolvimento.

Sobre o código gerado com apoio de IA

A IA foi utilizada de forma significativa durante a construção do protótipo.

Por esse motivo, antes da entrega, revisei a estrutura da aplicação, as principais funções de domínio, a integração com os dados e os testes para conseguir explicar a solução entregue.

As decisões arquiteturais e regras que devem ser preservadas também estão documentadas em:

- `../01-INCEPTION.md`
- `../02-PLAN.md`
- `../03-OPERATIONS.md`
- `../AGENTS.md`

Arquivos desta pasta

`01-chatgpt-case-painel-cliente.pdf`

Histórico cronológico da conversa utilizada como apoio para desenvolver este case.

Salve:

Command + S

Essa nota é boa porque você não tenta esconder:

"A IA foi utilizada de forma significativa."

Isso passa muito melhor do que tentar fazer o avaliador acreditar que não foi.

PASSO 4 — Agora precisamos salvar nossa conversa

Aqui está a parte que você estava preocupado.

Você não vai exportar conversas de Java, Shopee, faculdade etc.

Só queremos a conversa relacionada a:

Case Técnico – Painel do Cliente

Ou seja, esta conversa que começou quando você me mostrou o case e pediu ajuda para construí-lo.

PASSO 5 — Salvar esta conversa em PDF

No navegador, com esta conversa aberta, pressione:

Command + P

Vai abrir a tela de impressão do Mac.

Procure a opção:

PDF

e escolha:

Salvar como PDF

Utilize o nome:

01-chatgpt-case-painel-cliente.pdf

Salve inicialmente em:

Downloads

ou na sua Área de Trabalho.

Importante

O PDF deve manter a conversa em ordem.

Não fique apagando:

"não entendi"

ou:

```
"deu erro"
```

ou minhas respostas que você não gostou.

Na verdade isso ajuda a mostrar que houve interação:

```
IA respondeu
↓
você não entendeu
↓
você questionou
↓
IA mudou a abordagem
```

É exatamente o tipo de evolução que o case diz querer observar.

PASSO 6 — Colocar o PDF dentro do projeto

Depois de salvar o PDF, abra o Finder.

Vá até onde salvou:

```
01-chatgpt-case-painel-cliente.pdf
```

Copie.

Depois vá para:

```
Documents
↓
case-painel-cliente
↓
historico
```

Cole o PDF ali.

Agora no VS Code deverá aparecer:

```
historico/
|
├── 00-LEIA-ME.md
└── 01-chatgpt-case-painel-cliente.pdf
```

E se o PDF ficar enorme?

Não tem problema.

O case permite:

```
Markdown
JSON
PDF
texto
```

desde que seja legível e cronológico.

Se o navegador não conseguir gerar tudo em um único PDF, podemos dividir:

```
historico/
|
├─ 00-LEIA-ME.md
├─ 01-chatgpt-parte-1.pdf
├─ 02-chatgpt-parte-2.pdf
└─ 03-chatgpt-parte-3.pdf
```

Desde que a ordem fique clara.

PASSO 7 — Antes de fazer commit

No terminal:

```
git status
```

Esperamos algo semelhante a:

```
Untracked files:
  historico/
```

Perfeito.

Agora confirme que dentro da pasta estão:

```
00-LEIA-ME.md
01-chatgpt-case-painel-cliente.pdf
```

PASSO 8 — Adicionar ao Git

Digite:

```
git add historico/
```

Enter.

Depois:

```
git status
```

Queremos algo parecido com:

Changes to be committed:

```
new file: historico/00-LEIA-ME.md  
new file: historico/01-chatgpt-case-painel-cliente.pdf
```

PASSO 9 — Commit

Digite:

```
git commit -m "docs: adiciona historico de interacoes com IA"
```

Enter.

PASSO 10 — Subir para o GitHub

Digite:

```
git push
```

Enter.

Depois:

```
git status
```

Queremos:

```
nothing to commit, working tree clean
```

PASSO 11 — Conferir no GitHub

Atualize o GitHub.

Você deverá encontrar:

```
historico
```

Clique nela.

Deverá aparecer:

```
00-LEIA-ME.md  
01-chatgpt-case-painel-cliente.pdf
```

Pronto.

Sobre a sua preocupação de "vai dar para perceber que eu fiz tudo que a IA passou"

Sim, eles provavelmente vão perceber que houve bastante orientação da IA. E o case foi construído justamente para permitir e analisar isso.

O verdadeiro risco não é terem descoberto que a IA escreveu código. O risco seria você chegar na próxima fase e não conseguir explicar:

```
calcularPatrimonioTotal()  
  
calcularAlocacao()  
  
fetch()  
  
async / await  
  
renderizarCliente()  
  
identificarOportunidadeCrossSell()  
  
assert.equal()  
  
assert.throws()
```

porque eles também dizem que vão avaliar se você domina a própria solução e consegue fazer alteração em tempo real.

Por isso, **depois do** `/historico`, **eu não quero adicionar mais funcionalidades só para enfeitar o case.**

Nossa reta final deve ser:

HISTÓRICO



↓

REVISÃO FINAL DO README

↓

REVISÃO DE TODOS OS REQUISITOS

↓

APRENDER O CÓDIGO QUE VOCÊ ENTREGOU

↓

SIMULAR ENTREVISTA TÉCNICA

↓

SIMULAR ALTERAÇÃO AO VIVO

↓

ADICIONAR rodfercar

↓

ENTREGA

Essa parte de **aprender o código** agora é tão importante quanto terminar o GitHub. Depois que você subir a pasta `/historico`, eu sugiro que a gente pare de simplesmente copiar arquivos e faça uma revisão do seu projeto **arquivo por arquivo, função por função**, bem leiga, até você conseguir explicar tudo sozinho na entrevista.